
SMASH: Probing Speech Recognition Robustness via Semantically Targeted Bit Flips

Zafaryab Haider

ECE, University of Maine, ME, USA
zafaryab.haider@maine.edu

Md Hafizur Rahman

ECE, University of Maine, ME, USA
md.hafizur.rahman@maine.edu

Ayşegül Bumin *

Amazon, CA, USA
abbumin@amazon.com

Prabuddha Chakraborty †

ECE, University of Maine, ME, USA
prabuddha@maine.edu

Abstract

Automatic speech recognition (ASR) systems are usually judged by aggregate transcript accuracy, but real failures often hinge on meaning: a wrong dose, address, account number, or date can matter more than many harmless word errors. We introduce SMASH, a framework for finding targeted semantic failures caused by sparse INT8 weight-bit perturbations in quantized sequence-to-sequence ASR models. Given an audio input and a meaning-critical span, SMASH searches for small weight changes that replace the intended meaning while keeping the transcript readable, plausible, and close to the original output. Each accepted fault certifies that a semantic substitution is reachable under a bounded perturbation budget. We study numeric and scalar substitutions across Whisper-small.en (WS), Whisper-large-v3 (WLv3), and SeamlessM4T-v2-large (S-M4T) on three corpora: an LLM-assisted controlled corpus (LACC), MultiMed, and LibriSpeech. On LACC at a 20-bit budget (B), SMASH-hybrid accepts numeric substitutions on 24/45 WS targets, 19/44 WLv3 targets, and 5/34 S-M4T targets; reachability is lower on real-speech datasets. In matched LACC budget sweeps up to $B = 40$, WS reaches its ceiling by $B = 20$, while WLv3 and S-M4T continue to gain accepted substitutions through $B = 40$. A blinded, method-mixed 105-output validation pool audits semantic cleanliness: human-majority labels and a four-judge LLM ensemble agree on the composite human-clean label with $\kappa = 1.00$, while per-question agreement is lower. Faults are sparse: the pooled median is three INT8 bit flips, with medians of two on WS, eleven on WLv3, and twelve on S-M4T. SMASH shifts ASR robustness evaluation from transcript edit distance to targeted semantic reachability: which meanings can be changed, how plausibly, and with how few bit flips.

1 Introduction

Automatic speech recognition (ASR) robustness is usually reported as aggregate transcript accuracy, yet many deployment failures are span-local and semantic. In clinical dictation, emergency dispatch, financial transcription, administrative records, and accessibility tools, a changed number, address, identifier, date, or name can matter more than many harmless word errors. Changing “vaccine was stored at -20 ” to “vaccine was stored at -30 ” preserves fluency and most tokens, but changes an

*This work was performed in the author’s personal capacity. The views and opinions expressed in this paper are solely those of the author and do not reflect the views of Amazon.com, Inc. or its affiliates.

†Corresponding Author

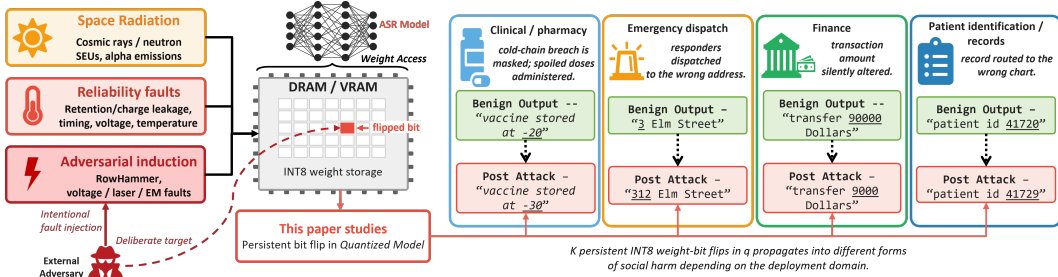


Figure 1: **Fault-to-transcript diagnostic pathway.** Sparse persistent perturbations of stored INT8 codes can arise from reliability faults or adversarial fault mechanisms. We use this setting as a pre-set-scoped diagnostic model to test whether bounded parameter faults can produce meaning-changing ASR transcripts. The figure motivates the robustness audit; it is not an end-to-end physical exploit or deployment-harm demonstration.

operational fact. Such failures are poorly captured by WER, CER, or likelihood, which measure how much a transcript moved rather than which facts can be changed.

We study this gap for quantized sequence-to-sequence ASR under sparse INT8 parameter faults. A flipped stored weight bit produces a discrete perturbation to one quantized parameter that can propagate through autoregressive decoding. We use this setting as a worst-case robustness probe, not as a physical exploit claim. The resulting question is semantic reachability: given clean audio and a declared source span, can a few weight-bit flips force a specific meaning-changing substitution while leaving the transcript readable, plausible, and close to the quantized baseline?

We introduce SMASH, a tokenization-aware framework for targeted semantic bit-flip search. SMASH constructs a declared replacement target, ranks candidate INT8 flips with a span-conditioned objective, and accepts a certificate only when decoding realizes the target under a four-gate cascade: transcript change, readability, proximity, and target realization. Each accepted fault is therefore a per-sample certificate of reachable semantic substitution, not transcript degradation.

Across Whisper-small.en, Whisper-large-v3, and SeamlessM4T-v2-large on LACC, MultiMed, and LibriSpeech, numeric/scalar reachability is concentrated but nonzero and model- and corpus-dependent. On LACC at $B = 20$, SMASH-hybrid finds 24/45, 19/44, and 5/34 numeric substitutions including single-bit cases; real-speech rates are lower. Matched non-SMASH baselines move or collapse transcripts but produce no verified targeted substitutions. Human-majority and four-judge LLM labels agree on `clean_success` with $\kappa = 1.00$.

Contributions: First, we define semantic-integrity reachability as an ASR robustness audit target: declared source→target span realization under bounded sparse quantized faults, distinct from WER drift or generic transcript movement. Second, we introduce SMASH, a span-conditioned INT8 bit-flip search that instantiates this audit target for seq2seq ASR. Third, we evaluate reachability across three ASR backbones, three corpora, and budget sweeps, showing that numeric/scalar reachability is nonzero but concentrated. Fourth, we pair the automatic G1–G4 strict certificate with a blinded, method-mixed human-clean validation pool and a verified-fault ledger for reproducible auditing. Finally, we show that closest adaptable movement/degradation baselines do not realize declared targets under matched gates, while explicitly scoping this as an event-alignment comparison rather than a claim of optimal targeted span search.

2 Background and Related Work

Sociotechnical ASR Risk and Semantic Evaluation: ASR increasingly mediates consequential workflows like clinical documentation, emergency dispatch, financial transcription, accessibility tools, and voice-driven interfaces. Aggregate metrics like WER do not distinguish harmless from high-cost substitutions, motivating severity-aware ASR evaluation [43, 6] and clinical studies showing recognition errors persisting into reviewed notes across terminology, speakers, and workflows [51, 24]. We probe which high-impact semantic substitutions are reachable under worst-case sparse perturbations of deployed quantized ASR models.

Hardware Bit Flips: Natural, Reliability, and Adversarial Sources: Sparse bit-level parameter faults are motivated by a literature spanning natural memory faults, runtime reliability errors, and adversarial fault injection. Cosmic-ray and alpha-particle soft errors in semiconductor memory have been documented for decades [52, 2, 38], and onboard ML for spacecraft has renewed interest in radiation robustness for deployed inference [16]. Retention variation and refresh leakage drive runtime DRAM faults [18, 13], while RowHammer and its successors, Half-Double, Blacksmith, and RowPress, demonstrate adversarially induced disturbance flips [14, 15, 12, 19]. Voltage and frequency manipulation extend the attack surface beyond DRAM [40, 21]. We use this combined literature to motivate sparse parameter faults as a robustness regime, not to claim an end-to-end physical exploit.

Quantized Neural-Network Bit-Flip Attacks: Adversarial robustness has historically studied input perturbations [39, 9, 20] with certified [44, 4] and weight-space [7, 45] extensions. Integer quantization [11] exposes deterministic bit-level effects on stored weights: PBS collapses quantized classifiers [34], DeepHammer connects bit flips to RowHammer at system scale [47], T-BFA and integer-programming variants target classifier labels [36, 1], transformer defenses analyze parameter-fault resilience [41, 22], and SilentStriker studies stealthy LLM degradation [46]. Their success events are classification failure or aggregate degradation, not decoded semantic substitution. Recent fault-injection work does steer semantics, but in other modalities, differentiable bit-level semantic steering of quantized vision-language image captioning [10] and jailbreak behavior in quantized language models [49], whereas SMASH targets declared source→target semantic span realization in seq2seq ASR under the **G1–G4** transcript gates.

Input-Side ASR Attacks and the SMASH Gap: ASR adversarial work has mostly perturbed the audio waveform [3, 29, 37, 48, 23], including Whisper-specific prefix-induced muting and task control [25, 33, 32]. Within transcripts, ITN and entity-correction systems target high-value semiotic classes, cardinals, dates, money, identifiers, named entities [28, 50, 42].

To our knowledge, no previous work defines this success event: declared source→target semantic span realization in seq2seq ASR under sparse quantized weight faults, with success requiring transcript change, readability, locality, and target realization. Existing bit-flip methods optimize different events, such as classifier collapse, classifier-label redirection, or LLM degradation. Thus, there is no established event-equivalent SOTA baseline; our comparisons use the closest adaptable bit-flip families under matched post-hoc gates.

3 Threat Model

We study semantic reachability under sparse faults in the quantized parameters of sequence-to-sequence ASR models.

Let f_θ denote an encoder–decoder ASR model with parameters θ , and let $\mathcal{D}$ denote a deterministic decoding procedure, such as greedy decoding or fixed-beam decoding with no sampling. For an audio input x , the model produces transcript: $\hat{y}(x; \theta) = \mathcal{D}(f_\theta(x))$. Unless otherwise stated, all experiments use the deterministic decoding configuration specified in Section 5: no sampling, beam size one, and maximum generation length 128.

Attacker’s Capability and Non-Goals: We consider a white-box worst-case evaluator with access to the quantized weights of a pre-trained sequence-to-sequence ASR model. The evaluator may flip at most B bits of the stored integer codes within a declared parameter subset $\mathcal{W} \subset \theta$, which we call a *preset*. In our experiments, presets correspond to compact, attribution-relevant submodules, for example, the last two decoder layers together with the language-modeling head, or the encoder adapter on SeamlessM4T, chosen before evaluation; specific preset definitions are listed in Section 5. No retraining, activation perturbation, input perturbation, prompt modification, or audio modification is permitted: the waveform x is delivered clean, and the only degree of freedom is the choice of weight-bit flips within $\mathcal{W}$ under the stated budget.

Quantized Baseline and Preset-Scoped INT8 Representation: Evaluation is anchored to the quantized model. For each targeted weight tensor $W \in \mathbb{R}^{m \times n}$ in the preset $\mathcal{W}$, we form signed INT8 codes $q \in \{-128, \dots, 127\}^{m \times n}$ using symmetric max-absolute scaling with rounding/saturation:

$$q = \text{clip}_{[-128, 127]}(\text{round}(W \oslash S)), \quad \tilde{W} = q \odot S, \quad (1)$$

Span-Conditioned Proposal Objective: We use a span-conditioned objective that separates preserving the surrounding transcript from changing the selected pivot. Let $\ell_t(\theta) = -\log p_\theta(y_{0,t} \mid x, y_{0,<t})$ be the teacher-forced negative log-likelihood of the anchor token at position t . The objective is:

$$\mathcal{L}_{\text{SMASH}}(\theta) = \frac{1}{|\mathcal{N}|} \sum_{t \in \mathcal{N}} \ell_t(\theta) - \alpha \frac{1}{|\mathcal{C} \setminus \mathcal{T}|} \sum_{t \in \mathcal{C} \setminus \mathcal{T}} \ell_t(\theta) + \gamma \mathcal{L}_{\text{tgt}}(\theta) \quad (3)$$

The first term preserves the anchor transcript outside the critical region. The second term enters with a negative sign; minimizing $\mathcal{L}_{\text{SMASH}}$ therefore *increases* the teacher-forced loss on the original pivot tokens, weakening the model’s preference for the source semantic span. The third term promotes the declared target realization. When token-level target labels are available, we use

$$\mathcal{L}_{\text{tgt}}(\theta) = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} -\log p_\theta(y_t^{\text{tgt}} \mid x, y_{0,<t}) \quad (4)$$

The target-promoting term is essential to strict reachability. Removing it substantially reduces strict success in matched ablations, reported in Appendix D.1. The objective in Equation 3 is not a success criterion: it only proposes directions in parameter space that may make the declared substitution more reachable. Every proposed fault is still decoded and evaluated under the gates.

Bit-Flip Ranking: For a candidate flip at scalar coordinate u and bit position b , let $\Delta q_u(b)$ be the induced change in the signed INT8 code and let S_u be the fixed quantization scale at that coordinate. The induced dequantized perturbation is $\Delta W_u(b) = S_u \Delta q_u(b)$. For any differentiable proposal objective $R(\theta)$, a first-order approximation gives $\Delta R \approx \frac{\partial R}{\partial W_u} \Delta W_u(b)$.

We use this estimate to rank candidate bit flips inside the preset $\mathcal{W}$ subject to the global budget B , the per-location cap K , and the bit-position constraints specified in Section 5. In the main configuration, $R = \mathcal{L}_{\text{SMASH}}$ from Equation 3, and candidates are scored by the predicted decrease in R . Because $\mathcal{L}_{\text{SMASH}}$ jointly preserves non-critical context and weakens the selected pivot while promoting the target, a single ranking signal trades off all three objectives. This recovers the first-order logic of progressive bit search but replaces the global cross-entropy objective with a semantic span-conditioned one.

Ranking Variants: The implementation supports a target logit-margin ranking variant, which scores the declared span substitution before softmax and may select different bits than teacher-forced loss; it is not part of the numeric/scalar core results reported here.

Bundled search over sparse faults: After ranking flips, SMASH tests small high-ranked bundles because isolated logit changes may not alter autoregressive decoding. Each step applies a bounded bundle, resynchronizes affected dequantized weights, decodes the perturbed model, and commits or reverts the bundle. Accepted bundles update F . Search stops at strict success, budget exhaustion, or the step limit. Certificate cost is committed flips $|F|$, not budget B . Sensitivity and hyperparameters appear in Appendix E.3 and Section 5.

Outcome Gates: We evaluate each faulted transcript $\hat{y}_F$ using a four-gate outcome definition. Gates **(G1)–(G3)** measure whether the output changes while remaining readable and close to the baseline. Gate **(G4)** measures whether the declared semantic target is realized. More details about the Gates are provided in the Appendix B. A *strict semantic success* $\text{Strict}(x, F) = \mathbf{1}\{\mathbf{G1} \wedge \mathbf{G2} \wedge \mathbf{G3} \wedge \mathbf{G4}\}$ is a fault configuration F for which all four gates hold. A sample is a strict success if some fault F within budget B passes all four gates.

Outcomes: We classify outcomes by gate failure or success: **No effect** fails **G1**; **degenerate/invalid** passes **G1** but fails **G2**; **plausibility-violating drift** passes **G1–G2** but fails **G3**; **low-impact movement** passes **G1–G3** but fails **G4**; and **strict semantic success** passes **G1–G4**. We report **source-disrupted partial** diagnostics when the source span is altered without modifying target.

5 Experimental Setup

Unless otherwise stated, all cells use deterministic decoding, fixed random seed 42, and $N = 100$ audio inputs.

Models: WS [30], WLv3 [31], and S-M4T [5]. All experiments use deterministic English decoding with no sampling, beam size one, and maximum generation length 128, so output changes are attributable to parameter faults rather than decoding stochasticity.

Datasets: LACC is a controlled TTS evaluation harness with manually curated pivot-controlled utterances; full construction details are in Appendix C. **MultiMed** [17] provides English medical-domain ASR with safety-relevant content such as medications, dosages, and units. **LibriSpeech** [26] is a clean-read benchmark used as a lower-pivot-density reference. Each cell uses $N=100$ utterances; we report both N and the target-constructible count $|T|$, since strict success is only defined for samples with a constructed target.

Quantization and parameter presets: All experiments use the preset-scoped INT8 quantization scheme from Section 3. WS and WLv3 use `whisper_decoder_last2_plus_lm_head`, covering the last two decoder layers and LM head; S-M4T uses `seamless_encoder_last2_plus_adapter`. Full tensor lists and quantization-only sanity checks are in Appendix A.

Canonical SMASH configuration: The main experiments use three hybrid SMASH configurations, WS-hybrid, WLv3-hybrid, and S-M4T-hybrid, each paired with its model-specific preset from above. All use automatic target construction, deterministic decoding, hybrid acceptance, `smash_decode_trials=2`, and `smash_gamma_target=10`. Headline reporting focuses on numeric/scalar pivots. Hybrid acceptance is a search strategy only: reported strict successes must still satisfy **G1–G4**. Candidate flips use bit positions 0..6, excluding the sign bit. We sweep $B \in \{1, 2, 3, 4, 5, 10, 20, 40\}$, with a LACC-only $B=100$ sidecar control, per-location cap $K = 3$, maximum 30 outer search steps, bundle size one, and $\tau_{\text{wer}} = 0.20$. Sensitivity analyses are in Appendix E.3. A decode-only WS-decode ablation, using the same WS preset but strict decode acceptance, is reported in Appendix D.2.

Baselines: We compare against non-SMASH sparse bit-flip baselines: targeted SilentStriker-native and untargeted gated/random families, including PBS-CE, PBS-CE-gated, SilentStriker-CE, SilentStriker-CE-gated, URBF, and TGRF variants. Whenever possible, baselines use the same model, dataset, sample set, seed, target-construction pipeline, and post-hoc gates. This makes transcript movement and targeted semantic substitution event-aligned: untargeted methods need not optimize the declared target, but they count as strict successes only if their outputs realize it under the same gates. The baseline grid includes PBS variants at $B \in \{20, 40\}$, SilentStriker variants at $B = 40$, and URBF/TGRF variants at $B = 20$. SilentStriker is reported at $B = 40$ because matched $B = 20$ runs were not available for that family. Full grids and movement decompositions are in Appendix F. Appendix F.1 specifies how classifier/LLM bit-flip baselines are adapted to seq2seq ASR, including objective, decode gate, and bit-position policy. We use each method’s native bit policy rather than forcing a matched policy: canonical SMASH excludes the INT8 sign bit, while PBS/URBF/TGRF allow it and SilentStriker-native is sign-bit-only. This makes the comparison conservative with respect to perturbation freedom, since SMASH searches a smaller per-weight bit pool yet is the only method that realizes strict semantic targets.

6 Results

We evaluate three questions: (i) are targeted semantic substitutions reachable under sparse INT8 weight faults, (ii) how does reachability vary across models and corpora, and (iii) does targeted semantic reachability differ measurably from generic transcript movement. Unless stated otherwise, rates are reported over target-constructible numeric samples T_{num} . We use *strict* for gate-cascaded target realization and *partial* for the diagnostic case in which **G1–G3** pass and the source pivot span is disrupted in the adversarial output; partial is not an exclusive bucket and overlaps with strict by construction. WER is reported against the quantized baseline transcript and is bucketed into *no effect* ($\text{WER} = 0$), *local edit* ($0 < \text{WER} \leq 0.20$), and *drift* ($\text{WER} > 0.20$).

Numeric reachability is concentrated and model-dependent: On LACC at $B=20$, SMASH-hybrid lands clean numeric substitutions on 24/45 WS, 19/44 WLv3, and 5/34 S-M4T pivots. The effect is therefore not a single-model artifact, but it is strongly model-dependent: WS is easiest, WLv3 intermediate, and S-M4T hardest under the same headline budget. The matched budget sweep in Figure 3 supports the same pattern: WS reaches its observed ceiling by $B = 20$, while WLv3 and S-M4T expose additional successes only at higher budgets.

Table 1: Numeric/scalar reachability for SMASH-hybrid. Strict requires G1–G4. Partial is strict-inclusive: G1–G3 plus either target realization or automatic source-span disruption. Drift denotes G1–G2 outputs failing locality. Wilson 95% CIs are in Appendix E.2.

Corpus	Model	$ T_{\text{num}} $	Strict@20	Strict@40	Δ	Partial@40	G1@40	Drift@40
LACC	WS	45	24/45	24/45	0	25/45	28/45	1/45
LACC	WLv3	44	19/44	22/44	+3	23/44	23/44	0/44
LACC	S-M4T	34	5/34	11/34	+6	24/34	29/34	3/34
MultiMed	WS	20	4/20	4/20	0	4/20	5/20	0/20
MultiMed	WLv3	26	2/26	4/26	+2	6/26	10/26	0/26
MultiMed	S-M4T	25	0/25	1/25	+1	6/25	10/25	0/25
LibriSpeech	WS	9	4/9	4/9	0	5/9	6/9	0/9
LibriSpeech	WLv3	8	0/8	1/8	+1	1/8	1/8	0/8
LibriSpeech	S-M4T	10	0/10	1/10	+1	1/10	2/10	0/10

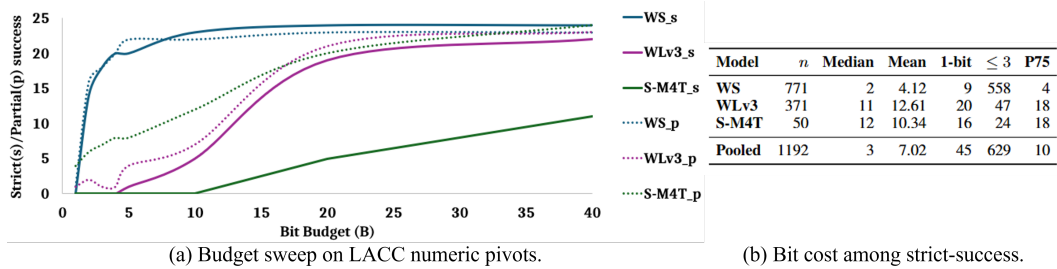


Figure 3: **(Left)** Solid curves show strict targeted successes under the full G_1 – G_4 gate cascade, while dotted curves show partial source-span disruption under G_1 – G_3 , without necessarily realizing the declared target. **(Right)** Realized bit-flip cost among strict-success certificates.

The real-speech corpora show lower but still informative reachability. Numeric pivots are denser in LACC than in MultiMed or LibriSpeech; the constructible sets are correspondingly smaller ($T_{\text{num}} = 20$ – 26 on MultiMed and 8 – 10 on LibriSpeech), and several real-speech cells reach zero strict successes. At $B = 20$, MultiMed yields $4/20$ strict successes on WS, $2/26$ on WLv3, and $0/25$ on S-M4T; LibriSpeech yields $4/9$, $0/8$, and $0/10$, respectively. We therefore treat LACC as a controlled probe of semantic reachability and the real-speech corpora as harder external checks. The zero-strict cells are not always inert. At $B = 40$, MultiMed/S-M4T reaches only $1/25$ strict success, but shows $6/25$ partial cases and $10/25$ decode changes. LACC/S-M4T similarly reaches $11/34$ strict successes but $24/34$ partial cases and $29/34$ decode changes. Thus, source-span disruption is often more reachable than declared-target realization, supporting the distinction between partial semantic movement and strict target reachability.

Strict certificates are sparse; cost is backbone-dependent: Across 1,192 strict-success certificates, the pooled median realized cost is three INT8 bit flips; single-bit certificates occur on every backbone, with 9 on WS, 20 on WLv3, and 16 on S-M4T; and 629/1,192 certificates (52.8%) commit at most three flips. The pooled statistic is dominated by WS volume. Backbone-conditioned medians are 2 for WS ($n = 771$), 11 for WLv3 ($n = 371$), and 12 for S-M4T ($n = 50$). Harder backbones therefore require denser realized flip sets when they succeed, but these are still sparse certificates rather than heavy or diffuse parameter perturbations.

Blinded human validation audits semantic cleanliness: On a 105-output method-mixed validation pool (Table 2), human-majority labels and a four-judge LLM ensemble agree on the composite human-clean label with $\kappa = 1.00$. This validates the composite audit decision on the sampled pool, not every automatic subgate globally: per-question agreement is lower, with $\kappa \in [0.51, 0.85]$.

We also run a supplemental LLM-only audit for cross-backbone SMASH-hybrid coverage. The four-judge ensemble labels 22/26 WS examples and 5/6 S-M4T examples as clean; combined with the original WLv3 22/26, the cross-model SMASH-hybrid clean rate is 49/58 (84.5%). These supplemental rows are not pooled with the shared human-validation set and are not used to compute

Table 2: Validation summary: The shared $n = 105$ set is the basis for the human–LLM agreement claim: human-majority labels and the four-judge LLM ensemble agree perfectly on the composite `clean_success` label, with $\kappa = 1.00$. The supplemental SMASH-hybrid rows are LLM-only and extend backbone coverage; they are not pooled with the shared human validation set and are not used to compute κ . Full per-question, per-judge, and overlap analyses are in Appendix H.

Set	Family / evidence	n	Clean	Not clean	Role
Shared human+LLM	SMASH-hybrid	26	22	4	WLv3 slice
Shared human+LLM	SMASH-decode	42	11	31	Cross-model ablation
Shared human+LLM	SS-native	7	0	7	Targeted non-SMASH
Shared human+LLM	PBS-CE-gated	16	0	16	Untargeted non-SMASH
Shared human+LLM	SS-CE-gated	6	0	6	Non-SMASH CE-gated
Shared human+LLM	TGRF-CE	8	0	8	Untargeted non-SMASH
Supplemental LLM-only	SMASH-hybrid WS	26	22	4	Backbone audit
Supplemental LLM-only	SMASH-hybrid WLv3	26	22	4	Existing validation slice
Supplemental LLM-only	SMASH-hybrid S-M4T	6	5	1	Backbone audit
Supplemental LLM-only	SMASH-hybrid total	58	49	9	WS + WLv3 + S-M4T

Table 3: Matched WS–LACC cross-method comparison. WER bands partition decoded outputs into no effect, local movement, and high-WER drift; Strict separately requires the full G1–G4 declared-target cascade. The table tests whether closest adaptable movement/degradation baselines realize declared semantic targets under matched post-hoc gates; it is not a denominator-matched comparison against a targeted span-CE or target-margin optimizer.

Method	Targeted?	B	N	No eff.	Local	Drift	Strict
SMASH (hybrid)	yes	20	100	59	40	1	26
SMASH (hybrid)	yes	40	100	57	42	1	26
PBS-CE	no	20	100	20	0	80	0
PBS-CE-gated	no	20	180	92	88	0	0
URBF	no	20	100	100	0	0	0
TGRF-CE	no	20	100	61	39	0	0
TGRF-MSB	no	20	100	98	2	0	0
TGRF-Uniform	no	20	100	100	0	0	0
PBS-CE	no	40	100	20	0	80	0
SilentStriker-native	yes	40	200	138	7	55	0
SilentStriker-CE	yes	40	100	10	6	84	0
SilentStriker-CE-gated	yes	40	200	188	12	0	0

human–LLM κ . Per-question agreement is lower than composite agreement, with gate-level κ values in the 0.51–0.85 range. We report per-question κ , per-judge breakdowns, and the held-out 24-item human-overlap sanity check in Appendix H. The main analysis uses the composite `clean_success` label because it matches the decision rule used to count accepted semantic substitutions.

Comparison scope: The cross-method comparison is an event-alignment diagnostic, not a direct SOTA horse race. Prior bit-flip methods optimize different events, so we adapt the closest available families and evaluate all outputs under the same post-hoc G1–G4 target-realization gates. The comparison asks whether movement- or degradation-oriented faults realize declared semantic targets; it does not claim optimality against a future targeted span-CE baseline designed for this event.

Movement-oriented baselines do not realize declared targets under matched gates: We compare SMASH-hybrid against four baseline families: PBS [35], SilentStriker [46], URBF (uniform random bit flips), and TGRF (top-grad random flips, with `_uniform/_msb/_ce` acceptance variants); each is adapted to seq2seq ASR per Appendix F.1.

In the matched WS–LACC comparison of Table 3, **every reported non-SMASH row lands zero strict targeted substitutions, despite extensive transcript movement**. The WER buckets show three distinct failure modes. Random or MSB-restricted baselines mostly do not move the transcript: URBF and TGRF-Uniform produce 100/100 no-effect outputs, TGRF-MSB 98/100. CE-driven

methods move transcripts but not toward the declared target: PBS-CE at $B=20$ sends 80/100 outputs into drift, SilentStriker-CE at $B=40$ sends 84/100, both with zero strict successes. Gated CE methods suppress collapse and concentrate outputs in the local band, e.g., PBS-CE-gated with 88/100 local edits, but still produce zero strict targeted substitutions. SMASH-hybrid converts movement into an aimed semantic substitution that survives the readability and plausibility cascade. For example, PBS-CE can move the transcript by collapse rather than targeted substitution, rewriting “Avery Chen joined the call and greeted everyone warmly” as “A B”; for an audio-matched five-method comparison on the same samples, see Table 21 in Appendix F.5. The SilentStriker family is configured for targeted attacks, but on this matrix the auto-target builder did not engage on any sample; the runs executed under the fallback objective. We report these rows as evidence about movement and drift, not as successful SilentStriker target construction.

Locality realism re-shapes the reachable set, with preset-dependent sensitivity: Software-side locality constraints substantially reshape strict reach on the harder backbones, but do not eliminate SMASH-hybrid on the easier preset. On WS/LACC, the unconstrained baseline 24/45 is preserved by page-local, rowhammer-row [14], cacheline-burst, blacksmith-burst [12], and collateral-noise, each also reaching 24/45 target-realized. RowPress [19] drops WS to 15/45, and Half-Double [15] to 22/45. The harder backbones are more placement-sensitive. WLv3/LACC swings from 4/44 under cacheline-burst to 17/44 under blacksmith-burst, a $4\times$ difference on the same corpus. S-M4T/LACC produces 0/34 strict successes under either burst mode, despite 9–17 decode changes and 9–14 partials. Locality is therefore not a binary kill-switch; it interacts with where each preset places useful committed flips in memory. We treat the software locality constraints as necessary-condition realism checks, not a complete physical-exploit simulation. We do not model DRAM row mapping [27], refresh dynamics [8], or cache coherence. Full per-mode tables are in Appendix G.

7 Conclusion

We introduced SMASH, a framework for measuring targeted semantic reachability under sparse INT8 weight-bit faults in quantized sequence-to-sequence ASR, and instantiated it on numeric/scalar substitutions across three ASR backbones and three corpora. Targeted numeric substitutions are reachable on all three backbones in the controlled LACC setting, with lower and more uneven reachability on real-speech corpora. The realized costs are sparse (pooled median three INT8 bit flips, with single-bit successes observed on every backbone), but strongly model- and corpus-dependent. Independent human-majority and four-judge LLM-ensemble labels agree on the composite gate outcome at $\kappa=1.00$, and non-SMASH bit-flip baselines move transcripts but do not realize the declared semantic target under the same gates. The central conclusion is: aggregate transcript metrics can miss sparse, plausible-looking substitutions on precisely the spans whose meaning matters most, so semantic-integrity reachability is a robustness criterion distinct from WER drift or decode movement.

8 Grant Acknowledgment

This work was supported by the U.S. Department of Energy, Office of Science, Basic Energy Sciences, under Award #DE-SC0026398

9 Disclaimer

This report was prepared as an account of work sponsored by an agency of the United States Government. Neither the United States Government nor any agency thereof, nor any of their employees, makes any warranty, express or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or any agency thereof. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.

References

- [1] Jiawang Bai, Baoyuan Wu, Yong Zhang, Yiming Li, Zhifeng Li, and Shu-Tao Xia. Targeted attack against deep neural networks via flipping limited weight bits. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=iKQAk8a2kM0>.
- [2] R. Baumann. Soft errors in advanced computer systems. *Design & Test of Computers, IEEE*, 22:258 – 266, 06 2005. doi: 10.1109/MDT.2005.69.
- [3] Nicholas Carlini and David Wagner. Audio adversarial examples: Targeted attacks on speech-to-text. In *Proceedings of the IEEE Security and Privacy Workshops (SPW)*, 2018.
- [4] Jeremy M. Cohen, Elan Rosenfeld, and J. Zico Kolter. Certified adversarial robustness via randomized smoothing. In *International Conference on Machine Learning*, 2019.
- [5] Seamless Communication, Loïc Barrault, Yu-An Chung, Mariano Cora Meglioli, David Dale, Ning Dong, Paul-Ambroise Duquenne, Hady Elsahar, Hongyu Gong, Kevin Heffernan, John Hoffman, Christopher Klaiber, Pengwei Li, Daniel Licht, Jean Maillard, Alice Rakotoarison, Kaushik Ram Sadagopan, Guillaume Wenzek, Ethan Ye, Bapi Akula, Peng-Jen Chen, Naji El Hachem, Brian Ellis, Gabriel Mejia Gonzalez, Justin Haaheim, Prangthip Hansanti, Russ Howes, Bernie Huang, Min-Jae Hwang, Hirofumi Inaguma, Somya Jain, Elahe Kalbassi, Amanda Kallet, Ilia Kulikov, Janice Lam, Daniel Li, Xutai Ma, Ruslan Mavlyutov, Benjamin Pelloquin, Mohamed Ramadan, Abinash Ramakrishnan, Anna Sun, Kevin Tran, Tuan Tran, Igor Tufanov, Vish Vogeti, Carleigh Wood, Yilin Yang, Bokai Yu, Pierre Andrews, Can Balioglu, Marta R. Costa-jussà, Onur Celebi, Maha Elbayad, Cynthia Gao, Francisco Guzmán, Justine Kao, Ann Lee, Alexandre Mourachko, Juan Pino, Sravya Popuri, Christophe Ropers, Safiyyah Saleem, Holger Schwenk, Paden Tomasello, Changhan Wang, Jeff Wang, and Skyler Wang. Seamless4t: Massively multilingual & multimodal machine translation, 2023. URL <https://arxiv.org/abs/2308.11596>.
- [6] Zachary Ellis, Jared Joselowitz, Yash Deo, Yajie Vera He, Anna Kalygina, Aisling Higham, Mana Rahimzadeh, Yan Jia, Ibrahim Habli, and Ernest Lim. WER is unaware: Assessing how ASR errors distort clinical understanding in patient facing dialogue. In *Proceedings of the 16th International Workshop on Spoken Dialogue System Technology*, pages 391–417, Trento, Italy, feb 2026. Association for Computational Linguistics. URL <https://aclanthology.org/2026.iwds-1.39/>.
- [7] Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-aware minimization for efficiently improving generalization. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=6Tm1mposlrM>.
- [8] Pietro Frigo, Emanuele Vannacci, Hasan Hassan, Victor van der Veen, Onur Mutlu, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. TRRespass: Exploiting the many sides of target row refresh. In *IEEE Symposium on Security and Privacy (S&P)*, 2020.
- [9] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*, 2015.
- [10] Zafaryab Haider, Md Hafizur Rahman, Shane Moeykens, Vijay Devabhaktuni, and Prabuddha Chakraborty. How a bit becomes a story: Semantic steering via differentiable fault injection. arXiv preprint arXiv:2512.14715, 2025.
- [11] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018. doi: 10.1109/CVPR.2018.00286.
- [12] Patrick Jattke, Victor van der Veen, Pietro Frigo, Stijn Gunter, and Kaveh Razavi. BLACK-SMITH: Scalable rowhammering in the frequency domain. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2022.

- [13] Samira Khan, Donghyuk Lee, Yoongu Kim, Alaa R. Alameldeen, Chris Wilkerson, and Onur Mutlu. The efficacy of error mitigation techniques for dram retention failures: a comparative experimental study. In *The 2014 ACM International Conference on Measurement and Modeling of Computer Systems, SIGMETRICS '14*, page 519–532, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450327893. doi: 10.1145/2591971.2592000. URL <https://doi.org/10.1145/2591971.2592000>.
- [14] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors. In *Proceedings of the ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2014.
- [15] Andreas Kogler, Jonas Juffinger, Salman Qazi, Yoongu Kim, Moritz Lipp, Nicolas Boichat, Eric Shiu, Mattias Nissler, and Daniel Gruss. Half-double: Hammering from the next row over. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3807–3824. USENIX Association, 2022.
- [16] Kevin Lange, Federico Fontana, Francesco Rossi, Mattia Varile, and Giovanni Apruzzese. Machine learning in space: Surveying the robustness of on-board ml models to radiation. In *2024 IEEE Space Computing Conference (SCC)*, pages 51–64. IEEE, 2024.
- [17] Khai Le-Duc, Tuyen Tran, Bach Phan Tat, Nguyen Kim Hai Bui, Quan Dang, Hung-Phong Tran, Thanh-Thuy Nguyen, Ly Nguyen, Tuan-Minh Phan, Thi Thu Phuong Tran, Chris Ngo, Nguyen X. Khanh, and Thanh Nguyen-Tang. Multimed-st: Large-scale many-to-many multilingual medical speech translation, 2025. URL <https://arxiv.org/abs/2504.03546>.
- [18] Jamie Liu, Ben Jaiyen, Richard Veras, and Onur Mutlu. Raidr: Retention-aware intelligent dram refresh. *ACM SIGARCH Computer Architecture News*, 40(3):1–12, 2012.
- [19] Haocong Luo, Ataberk Olgun, A. Giray Yağlıkcı, Yahya Can Tuğrul, Steve Rhyner, Meryem Banu Cavlak, Joël Lindegger, Mohammad Sadrosadati, and Onur Mutlu. RowPress: Amplifying read disturbance in modern DRAM chips. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–18, 2023. doi: 10.1145/3579371.3589063.
- [20] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations*, 2018.
- [21] Kit Murdock, David Oswald, Flavio D Garcia, Jo Van Bulck, Daniel Gruss, and Frank Piessens. Plundervolt: Software-based fault injection attacks against intel sgx. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1466–1482. IEEE, 2020.
- [22] Najmeh Nazari, Hosein Mohammadi Makrani, Chongzhou Fang, Hossein Sayadi, Setareh Rafatirad, Khaled N. Khasawneh, and Houman Homayoun. Forget and rewire: Enhancing the resilience of transformer-based models against bit-flip attacks. In *33rd USENIX Security Symposium*. USENIX Association, 2024. URL <https://www.usenix.org/conference/usenixsecurity24/presentation/nazari>.
- [23] Paarth Neekhara, Shehzeen Hussain, Prakhar Pandey, Shlomo Dubnov, Julian McAuley, and Farinaz Koushanfar. Universal adversarial perturbations for speech recognition systems. In *Interspeech 2019*, pages 481–485, 2019. doi: 10.21437/Interspeech.2019-1353.
- [24] Joel Jia Wei Ng, Eugene Wang, Xinyan Zhou, Kevin Xiang Zhou, Charlene Xing Le Goh, Gabriel Zheng Ning Sim, Hiang Khoon Tan, Serene Si Ning Goh, and Qin Xiang Ng. Evaluating the performance of artificial intelligence-based speech recognition for clinical documentation: A systematic review. *BMC Medical Informatics and Decision Making*, 25(236), 2025. doi: 10.1186/s12911-025-03061-0. URL <https://doi.org/10.1186/s12911-025-03061-0>.
- [25] Raphael Olivier and Bhiksha Raj. There is more than one kind of robustness: Fooling whisper with adversarial examples. In *Interspeech 2023*, pages 4394–4398, 2023. doi: 10.21437/Interspeech.2023-1105.

- [26] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. Librispeech: An asr corpus based on public domain audio books. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5206–5210, 2015. doi: 10.1109/ICASSP.2015.7178964.
- [27] Peter Pessl, Daniel Gruss, Clémentine Maurice, Michael Schwarz, and Stefan Mangard. {DRAMA}: Exploiting {DRAM} addressing for {Cross-CPU} attacks. In *25th USENIX security symposium (USENIX security 16)*, pages 565–581, 2016.
- [28] Ernest Pusateri, Bharat Ram Ambati, Elizabeth Brooks, Ondrej Plátek, Donald McAllaster, and Venki Nagesha. A mostly data-driven approach to inverse text normalization. In *Interspeech 2017*, pages 2784–2788, 2017. doi: 10.21437/Interspeech.2017-1274.
- [29] Yao Qin, Nicholas Carlini, Garrison Cottrell, Ian Goodfellow, and Colin Raffel. Imperceptible, robust, and targeted adversarial examples for automatic speech recognition. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2019.
- [30] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision, 2022. URL <https://arxiv.org/abs/2212.04356>.
- [31] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *International Conference on Machine Learning*, 2023.
- [32] Vyas Raina and Mark Gales. Controlling whisper: Universal acoustic adversarial attacks to control speech foundation models. *arXiv preprint arXiv:2407.04482*, 2024. URL <https://arxiv.org/abs/2407.04482>.
- [33] Vyas Raina, Rao Ma, Charles McGhee, Kate Knill, and Mark Gales. Muting whisper: A universal acoustic adversarial attack on speech foundation models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 7549–7565, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.430. URL <https://aclanthology.org/2024.emnlp-main.430/>.
- [34] Adnan Siraj Rakin, Zhezhi He, and Deliang Fan. Bit-flip attack: Crushing neural network with progressive bit search. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [35] Adnan Siraj Rakin, Zhezhi He, and Deliang Fan. Bit-flip attack: Crushing neural network with progressive bit search. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1211–1220, 2019.
- [36] Adnan Siraj Rakin, Zhezhi He, Jingtao Li, Fan Yao, Chaitali Chakrabarti, and Deliang Fan. T-BFA: Targeted bit-flip adversarial weight attack. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(11):7928–7939, 2021. doi: 10.1109/TPAMI.2021.3112932.
- [37] Lea Schönherr, Katharina Kohls, Steffen Zeiler, Thorsten Holz, and Dorothea Kolossa. Adversarial attacks against automatic speech recognition systems via psychoacoustic hiding. In *Network and Distributed System Security Symposium*, 2019. URL <https://www.ndss-symposium.org/ndss-paper/adversarial-attacks-against-automatic-speech-recognition-systems-via-psychoacoustic-hiding/>.
- [38] Bianca Schroeder, Eduardo Pinheiro, and Wolf-Dietrich Weber. Dram errors in the wild: A large-scale field study. In *SIGMETRICS*, 2009.
- [39] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *International Conference on Learning Representations*, 2014.
- [40] Adrian Tang, Simha Sethumadhavan, and Salvatore Stolfo. {CLKSCREW}: Exposing the perils of {Security-Oblivious} energy management. In *26th USENIX Security Symposium (USENIX Security 17)*, pages 1057–1074, 2017.

- [41] Jialai Wang, Ziyuan Zhang, Meiqi Wang, Han Qiu, Tianwei Zhang, Qi Li, Zongpeng Li, Tao Wei, and Chao Zhang. Aegis: Mitigating targeted bit-flip attacks against deep neural networks. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 2329–2346, 2023.
- [42] Yi-Cheng Wang, Hsin-Wei Wang, Bi-Cheng Yan, Chi-Han Lin, and Berlin Chen. DANCER: Entity description augmented named entity corrector for automatic speech recognition. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, pages 4333–4342, Torino, Italia, 2024. ELRA and ICCL. URL <https://aclanthology.org/2024.lrec-main.387/>.
- [43] Ryan Whetten and Casey Kennington. Evaluating and improving automatic speech recognition using severity. In *Proceedings of the 22nd Workshop on Biomedical Natural Language Processing and BioNLP Shared Tasks*, pages 79–91, Toronto, Canada, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.bionlp-1.6. URL <https://aclanthology.org/2023.bionlp-1.6/>.
- [44] Eric Wong and J. Zico Kolter. Provable defenses against adversarial examples via the convex outer adversarial polytope. In *International Conference on Machine Learning*, 2018.
- [45] Dongxian Wu, Shu-Tao Xia, and Yisen Wang. Adversarial weight perturbation helps robust generalization. In *Advances in Neural Information Processing Systems*, 2020.
- [46] Haotian Xu, Qingsong Peng, Jie Shi, Huadi Zheng, Yu Li, and Cheng Zhuo. Silentstriker: Toward stealthy bit-flip attacks on large language models. *arXiv preprint arXiv:2509.17371*, 2025.
- [47] Fan Yao, Adnan Siraj Rakin, and Deliang Fan. Deephammer: Depleting the intelligence of deep neural networks through targeted chain of bit flips. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 1463–1480, 2020.
- [48] Xuejing Yuan, Yuxuan Chen, Yue Zhao, Yunhui Long, Xiaokang Liu, Kai Chen, Shengzhi Zhang, Heqing Huang, Xiaofeng Wang, and Carl A. Gunter. CommanderSong: A systematic approach for practical adversarial voice recognition. In *27th USENIX Security Symposium*, pages 49–64. USENIX Association, 2018. URL <https://www.usenix.org/conference/usenixsecurity18/presentation/yuan-xuejing>.
- [49] Noureldin Zahran, Ahmad Tahmasivand, Ihsen Alouani, Khaled Khasawneh, and Mohammed E. Fouda. On jailbreaking quantized language models through fault injection attacks. *arXiv preprint arXiv:2507.03236*, 2025.
- [50] Yang Zhang, Evelina Bakhturina, Kyle Gorman, and Boris Ginsburg. NeMo inverse text normalization: From development to production. In *Interspeech 2021*, pages 4468–4472, 2021. doi: 10.21437/Interspeech.2021-1571.
- [51] Li Zhou, Suzanne V. Blackley, Leigh Kowalski, Raymond Doan, Warren W. Acker, Adam B. Landman, Evgeni Kontrient, David Mack, Marie Meteer, David W. Bates, and Foster R. Goss. Analysis of errors in dictated clinical documents assisted by speech recognition software and professional transcriptionists. *JAMA Network Open*, 1(3):e180530, 2018. doi: 10.1001/jamaneetworkopen.2018.0530.
- [52] J. F. Ziegler and W. A. Lanford. Effect of cosmic rays on computer memories. *Science*, 206(4420):776–788, 1979. doi: 10.1126/science.206.4420.776.

A Quantization Details and Parameter Presets

This appendix gives the quantization and parameter-surface details used by the threat model and experimental setup. The goal is to make the perturbation domain explicit: SMASH does not perturb arbitrary floating-point weights, but operates on stored INT8 codes inside a declared parameter preset $\mathcal{W}$.

A.1 INT8 quantization

We use the preset-scoped INT8 quantization defined in eq. (1) of Section 3; here we record the implementation-side specifics that fix the perturbation domain. For 2D weights, scales are computed per output row ($s_i = \max_j |W_{ij}|/127$); for non-2D tensors, a single tensor-level scale ($s = \max |W|/127$) is broadcast over the tensor. Rows whose maximum absolute value is zero are assigned $s_i = 1$ to avoid division by zero; the corresponding codes are uniformly zero, so this case is degenerate by construction. The scale tensor S is computed once at quantization time and is not modified by the perturbation search: faults change only the integer code tensor q , while $\widetilde{W} = S \odot q$ is recovered through the fixed scales. Parameters outside the declared preset $\mathcal{W}$ remain at their full-precision values.

Table 4: **INT8 quantization scheme.** Two-dimensional weights use rowwise max-absolute scaling. Non-two-dimensional tensors use a single tensor-level max-absolute scale broadcast over the tensor.

Tensor type	Scale definition	Dequantization
2D weight matrix	$s_i = \max_j W_{ij} /127, S_{ij} = s_i$	$\widetilde{W}_{ij} = S_{ij} q_{ij}$
Non-2D weight tensor	$s = \max W /127, S \equiv s$	$\widetilde{W} = S \odot q$
Zero-scale case	scale set to 1	avoids division by zero
Code range	$q = \text{clip}(\text{round}(W/S), -128, 127)$	signed INT8 codes

A.2 Bit-flip semantics

A candidate perturbation flips one bit in one stored INT8 code. The implementation interprets the signed code q through its unsigned 8-bit view $q_u = q \bmod 256$ (equivalently, $q \ \& \ 0xFF$ in code), toggles bit b by XOR with 2^b , and remaps the result back to the signed range $[-128, 127]$:

$$q'_u(b) = (q_u \oplus 2^b), \quad q'(b) = \begin{cases} q'_u(b) - 256 & \text{if } q'_u(b) \geq 128, \\ q'_u(b) & \text{otherwise,} \end{cases} \quad \Delta q(b) = q'(b) - q.$$

The induced dequantized perturbation at scalar coordinate u is

$$\Delta W_u(b) = S_u \Delta q_u(b),$$

where S_u is the fixed quantization scale at u from §A.1; only q_u is touched by a flip, so the dequantized perturbation is linear in the pre-computed scale. Canonical SMASH restricts $b \in \{0, \dots, 6\}$ and excludes the sign bit, with per-location cap $K = 3$, so no scalar INT8 coordinate may absorb more than three bit flips in reported canonical runs; this policy and cap are summarized in Table 5. Untargeted baselines (e.g., PBS, random) may permit the sign bit and use different caps; their bit policies are reported with the corresponding baseline appendix.

A.3 Parameter-preset filtering

Each run restricts writes to a declared parameter preset $\mathcal{W}$. The selector first filters to valid weight tensors: parameters must be non-empty, end in `.weight`, and must not be biases or layer-normalization parameters. The preset then selects an architecture-specific subset of those valid weights.

For Whisper-small.en and Whisper-large-v3, the canonical preset is

`whisper_decoder_last2_plus_lm_head.`

It selects the last two decoder layers and the output head candidate used by the model implementation. The selector treats `proj_out.weight`, `lm_head.weight`, and decoder embedding/output-tied weights as output-head candidates when present.

Table 5: **Canonical bit-flip policy.** The main SMASH-hybrid runs flip stored INT8 codes under a fixed bit-position policy and per-location cap. Baselines may use different policies, which are reported separately.

Quantity	Canonical setting
Stored representation	signed INT8 code $q \in [-128, 127]$
Bit interpretation	two’s-complement unsigned toggle, then remap to signed INT8
Allowed bit positions	$\{0, 1, 2, 3, 4, 5, 6\}$
Sign bit	excluded in canonical SMASH
Per-location cap	$K = 3$
Dequantized perturbation	$\Delta W_u(b) = S_u \Delta q_u(b)$
Scale update	scales are fixed during perturbation search

Table 6: **Canonical parameter presets.** Each preset is applied after filtering to valid weight tensors: non-empty parameters ending in `.weight`, excluding biases and layer-normalization weights.

Model family	Preset	Selected parameter surface
WS / WL	<code>whisper_decoder_last2_plus_lm_head</code>	last two decoder layers plus output-head / output-embedding candidate
S-M4T	<code>seamless_encoder_last2_plus_adapter</code>	last two speech-encoder layers, speech adapter, and shared embedding when present

For SeamlessM4T-v2-large, the canonical preset is

`seamless_encoder_last2_plus_adapter.`

It selects the last two layers of `speech_encoder.encoder`, the speech-encoder adapter weights, and the shared embedding matrix when present. This preset is intentionally acoustic-side: it probes whether sparse perturbations to the speech-encoding and adapter surface can induce semantic substitutions in the generated text.

A.4 Preset sanity checks

If a recognized preset matches zero tensors, the runner raises an error rather than silently falling back to all weights. This prevents accidental experiments on an unintended parameter surface. The selected tensor names are logged with each run cell through the command, metadata, and per-sample artifacts. The compact verified-fault ledger stores the preset name and cell directory, while per-flip tensor names and coordinates are retained in the per-sample JSONL records described in Appendix I.

The preset definitions are therefore part of the measurement protocol: cross-model differences should be read as differences under the stated parameter surfaces, not as claims about unconstrained access to every model weight.

B Outcome Gates and Strict-Rate Definition

This appendix gives the explicit math for the gate cascade summarized in Sections 3 and 4. We evaluate each faulted transcript $\hat{y}_F$ using a four-gate outcome definition. Gates **G1–G3** measure whether the output changes while remaining readable and close to the quantized baseline. Gate **G4** measures whether the declared semantic target is realized.

G1 Decode change. The normalized faulted transcript differs from the normalized quantized baseline:

$$\text{norm}(\hat{y}_F) \neq \text{norm}(y_0). \quad (5)$$

Normalization removes punctuation, casing, whitespace variation, and equivalent numeric formatting.

G2 Readability. The output passes an automatic readability and degeneracy guard: it is non-empty, has bounded length ratio relative to y_0 , and contains no extreme character-level or word-level

repetition loops. This is an automatic well-formedness proxy, not a claim of human-perceived naturalness.

G3 Transcript proximity. The output remains close to the quantized baseline under edit distance:

$$\text{WER}(\hat{y}_F, y_0) \leq \tau_{\text{wer}}. \quad (6)$$

In all main experiments, $\tau_{\text{wer}} = 0.20$. This gate excludes global transcript collapse while allowing localized semantic drift.

G4 Target realization. For target-constructible samples with target tuple $\tau(x) = (k, s, r)$, the faulted transcript realizes the declared target r in the aligned neighborhood of the source span s , according to pivot-class-specific canonicalization and alignment rules:

$$\text{Hit}_k(\hat{y}_F; y_0, s, r) = 1. \quad (7)$$

Here Hit_k canonicalizes the relevant surface forms for class k —for example, numeric-form equivalence for numeric pivots, entity-surface matching for named entities, and polarity matching for negation—and requires the target realization to be localized to the selected semantic pivot rather than appearing only elsewhere in the transcript. Localization is enforced by requiring the canonicalized target r to occur in the alignment window of s , computed by token-level alignment against y_0 ; targets appearing only outside this window are not counted as hits. The pivot-specific hit rules are detailed in Section 4. This gate is evaluated for any method on matched target-constructible samples; targeted methods optimize for it, untargeted baselines do not.

A *strict semantic success* is a fault configuration F for which all four gates hold:

$$\text{Strict}(x, F) = \mathbf{1}\{\mathbf{G1} \wedge \mathbf{G2} \wedge \mathbf{G3} \wedge \mathbf{G4}\}. \quad (8)$$

For a set of target-constructible samples T , we report strict rate as

$$\text{StrictRate} = \frac{1}{|T|} \sum_{x \in T} \mathbf{1}\{\exists F : |F| \leq B, \text{Strict}(x, F) = 1\}. \quad (9)$$

This gate structure makes the comparison to untargeted baselines well-defined. Baselines may achieve decode movement or high-WER drift without satisfying the target-realization gate. Conversely, a baseline can only count as a strict semantic success if it realizes the same declared target under the same gates.

C LACC Construction

LACC is a 100-utterance controlled text-to-speech (TTS) evaluation harness designed to expose targeted semantic substitutions on four pivot classes. This appendix accompanies the references in Section 5 and documents motivation, composition, construction and validation protocol, pivot-density statistics, TTS configuration, corpus artifacts, and scope. LACC is not a natural-speech benchmark; real-speech evidence is reported separately on MultiMed and LibriSpeech.

C.1 Motivation: why a controlled corpus is needed

Targeted semantic-reachability evaluation requires four conditions that no single public ASR corpus jointly satisfies. **(i) Pivot density.** Real-speech corpora used in this work yield small target-constructible counts ($|T_{\text{num}}| \in \{8, 9, 10\}$ on LibriSpeech, 20–26 on MultiMed; see Appendix E.2), which limits budget-conditioned plateau analysis and produces wide Wilson confidence intervals. **(ii) Decode-baseline cleanliness.** Strict success is defined relative to the quantized baseline transcript y_0 . If y_0 already mis-transcribes the pivot span, the strict-rate event becomes confounded with quantization-only error rather than fault realization. Real-speech recordings carry acoustic noise, accent variation, and speaker effects that can perturb y_0 on the pivot span itself, blurring the attack-success signal. **(iii) Pivot purity.** Attributing observed reachability differences to pivot class requires clean separation between numeric-only, entity-only, negation-only, and mixed pivots. Natural utterances rarely admit such a partitioning by construction, so a real-speech-only design would entangle pivot-class effects with utterance-level confounds. **(iv) Reproducibility without**

audio redistribution. Public ASR corpora with high-density safety-relevant pivots are commonly encumbered by redistribution restrictions; a controlled-TTS corpus can be regenerated from its text and a TTS pipeline, removing that licensing barrier.

LACC is not a substitute for natural-speech evaluation. Its purpose is to provide a small, controlled, deterministic-prosody anchor on which targeted-reachability claims can be measured cleanly; real-speech generalization is then assessed separately on MultiMed and LibriSpeech.

C.2 Composition

The corpus contains 100 utterances, evenly split across four pivot categories:

- `numeric_only` (25 rows): a single numeric pivot, e.g. “*She took 5 mg of ibuprofen before bed last night*” with `pivot_span="5 mg"`.
- `entity_only` (25 rows): a single named-entity pivot (person or organization), e.g. “*Avery Chen joined the call and greeted everyone warmly*” with `pivot_span="Avery Chen"`.
- `negation_only` (25 rows): a single negation marker, e.g. “*I did not hear the last word clearly over the noise*” with `pivot_span="did not"`.
- `mixed_all_three` (25 rows): a contiguous span containing one element from each of the other three classes, e.g. “*Avery Chen did not take 5 mg of aspirin after lunch*” with `pivot_span="Avery Chen did not take 5 mg"`.

Each row carries fields `id`, `category`, `pivot_type`, `pivot_span`, `reference`, and `audio_path`; the manifest is `datasets/synthetic/synthetic_wa.jsonl`.

C.3 Construction and validation protocol

References were authored by hand and not LLM-generated; the “LLM-assisted” label refers to seed prompting during early drafting only. An LLM was used informally to elicit candidate sentence shells per category; the authors manually verified a small number of these candidates and then rewrote or discarded the rest before any audio was rendered. The corpus prose in `synthetic_wa.jsonl` is the post-rewrite output, not the LLM-elicited material; no specific model identifier or prompt template is therefore declared.

Each utterance was reviewed to satisfy three constraints: (i) the `pivot_span` appears as a contiguous substring of `reference`; (ii) the pivot is embedded in a complete sentence with surrounding context tokens, so that plausibility (**G3**) and proximity (τ_{wer}) gates remain meaningful; and (iii) for the three single-class categories, no out-of-class pivot is present (e.g., `numeric_only` samples contain no negation markers). Length is constrained to short clauses so that TTS prosody is uniform and search budgets are comparable across utterances.

Validation is a manual review pass on every candidate against the three constraints above, plus a check for duplicate `pivot_span` reuse that would compromise category purity, plus a TTS-comprehensibility check that the rendered audio is decodable to text close to the reference under all three backbones in the unperturbed quantized configuration. No automated named-entity-recognition or negation-detection tool was applied; constraint enforcement is fully manual by design, since the corpus is small enough that a single review pass per row is feasible and avoids tool-induced false positives on category purity. Candidates that failed any check were rewritten and re-reviewed; the final corpus is the post-review output.

C.4 Pivot-density statistics

- Reference length: 9–13 words (mean 10.2), 41–73 characters (mean 56.5).
- Single pivot span per utterance.
- Pivot uniqueness: 93 unique `pivot_span` values across 100 rows; the seven repeats are common negation markers (e.g., *did not*, *can’t*) reused across distinct sentences.
- Pivot length: 1–3 tokens for entity and negation pivots; 1–4 tokens for numeric pivots (digit + optional unit such as *mg*, *ml*, *degrees Celsius*, *bpm*); 5–10 tokens for mixed pivots, which span all three classes.

The numeric/scalar subset used in the main study covers the 25 `numeric_only` rows plus the numeric component of `mixed_all_three`; target-constructible counts $|T_{\text{num}}|$ per backbone differ slightly because the automatic target-construction procedure (Section 4) requires the source span to be token-alignable under each backbone’s tokenizer.

C.5 TTS configuration

Audio is generated by `src/TTS.py` using the Coqui TTS Python API with the model `tts_models/en/ljspeech/vits` (single-speaker English VITS trained on LJSpeech). Decoding is deterministic. Each utterance is rendered to one mono 16-bit PCM WAV at 16 kHz, matching the input sample rate expected by `Whisper-small.en`, `Whisper-large-v3`, and `SeamlessM4T-v2-large` with no further resampling. Audio durations span 2.82–7.62 seconds (mean 4.49, median 4.31); the total corpus duration is approximately 7.5 minutes. Audio files are written to `datasets/synthetic/audio/{id}.wav`.

For reproducibility, the TTS pipeline used the Coqui TTS package (TTS==0.22.0) with model `tts_models/en/ljspeech/vits`. The canonical invocation is:

```
python -m src.TTS -in <seed.jsonl> -out datasets/synthetic/synthetic_wa.jsonl
-audio_dir datasets/synthetic/audio -backend coqui -model
tts_models/en/ljspeech/vits
```

Rendered audio is converted to 16 kHz mono with `ffmpeg`:

```
ffmpeg -i <in.wav> -ac 1 -ar 16000 <out.wav>
```

The WAV files are therefore stored in the same 16 kHz mono format used for evaluation.

C.6 Corpus artifacts

The LACC corpus comprises three artifacts: (i) the JSONL manifest `datasets/synthetic/synthetic_wa.jsonl` with one row per utterance carrying `id`, `category`, `pivot_type`, `pivot_span`, `reference`, and `audio_path`; (ii) rendered 16 kHz mono 16-bit PCM WAV files at `datasets/synthetic/audio/{id}.wav` for the full 100-utterance set (≈ 7.5 minutes, ≈ 15 MB uncompressed); and (iii) the generation script `src/TTS.py` with the canonical invocation listed above, which renders the full corpus from the manifest. The public corpora MultiMed and LibriSpeech are used through their own public sources and are identified per cell by sample manifest.

C.7 Scope and limitations

LACC is a controlled-prosody, single-speaker TTS harness. Recording conditions, acoustic variability, accent diversity, and speaker variation are absent by design. The TTS backbone is fixed (Coqui VITS / LJSpeech); changing the synthesizer would change the prosody distribution presented to the ASR backbones and could shift the per-cell strict-rate ceiling. LACC results should be read as controlled reachability evidence on rendered audio, not as natural-speech generalization. Real-speech evidence is reported separately on MultiMed (medical-domain ASR with safety-relevant numeric and entity content) and LibriSpeech (clean-read benchmark, lower pivot density).

D Method Ablations

This appendix collects the method-side ablations referenced in Section 4. §D.1 is the controlled span-conditioned objective ablation that supports the main-text claim that target construction and target promotion are load-bearing. §D.2 is an acceptance-mode configuration reference, not a controlled ablation; the caveat is repeated at the head of that subsection because the two acceptance arms differ on several search and ranking knobs in addition to acceptance mode.

D.1 Span-Conditioned Objective Ablation

This appendix accompanies the reference in Section 4. We ablate the target-enabled component of the span-conditioned objective in the v27 implementation and report its effect on decoded outcomes.

Because the target-off arm disables target construction entirely, this appendix reports raw outcome counts over $N=100$ rather than StrictRate over the target-constructible denominator $|T|$.

D.1.1 Setup

The span-conditioned objective in Equation 3 combines a non-critical preservation term, a negative-weighted loss on the original pivot context, and a target-promotion term when a target label is available. In the v27 implementation, the closest valid target-off arm is obtained by setting `-smash_target_enable 0`. This disables target construction and therefore conditionally removes the $\gamma \cdot \text{target_CE}$ contribution while leaving the masked α -break objective active. There is no separate `ce` setting for `-smash_mask_style`, so this is the nearest implementation-faithful ablation of the target-promotion pathway.

We evaluate this ON/OFF contrast on Whisper-small.en $\times$ LACC at $B \in \{5, 20\}$ with $N=100$, seed 42, `max_steps=30`, hybrid acceptance, and $\gamma_{\text{target}} = 20$ in the target-enabled arm. The canonical target-enabled arm constructs targets on $|T| = 92$ samples. The target-off arm constructs no targets by design, so $|T| = 0$ for that arm and StrictRate is undefined. For this reason, Table 7 reports raw counts over $N=100$ rather than rates over $|T|$.

D.1.2 Result

Table 7 reports the matched outcomes. Disabling target construction and target promotion reduces strict decoded successes at both budgets and shifts the movement profile toward fewer local edits and more high-WER drift at $B=20$.

Table 7: **Span-conditioned objective ablation on WS $\times$ LACC.** Counts are over $N=100$ samples. The target-off arm disables target construction with `-smash_target_enable 0`, so $|T| = 0$ and StrictRate is undefined for that arm. $|F|$ median is computed over strict-success samples only.

Configuration	B	Strict	Local	Drift	$ F $ Median
(target_enable=1)	5	22	43	7	2
(target_enable=0)	5	6	32	7	3
(target_enable=1)	20	25	44	1	2
(target_enable=0)	20	9	21	7	3

At $B=5$, strict count drops from 22 to 6. At $B=20$, strict count drops from 25 to 9. The target-off arm therefore recovers only a minority of the strict events found by the target-enabled configuration. Median realized cost over strict-success samples also increases from 2 to 3 flips, indicating that the target-off search requires more committed perturbations even when it reaches a strict decoded event. At $B=20$, local edits fall from 44 to 21, while high-WER drift rises from 1 to 7, consistent with source disruption becoming less anchored to a declared target.

D.1.3 Interpretation

This ablation should be read as an implementation-faithful test of whether target construction and target promotion are load-bearing in v27. It is not a denominator-matched StrictRate comparison, because disabling `smash_target_enable` also disables construction of the target tuple used to define $|T|$. The appropriate comparison is therefore the raw decoded outcome profile over the same $N=100$ samples.

Under that comparison, the target-enabled objective is clearly more effective. The masked preserve-and-break structure alone can still produce some strict decoded events, but it finds far fewer of them, with higher realized cost and more drift. This supports the main methodological claim: SMASH is not merely an untargeted source-disruption search. Its target-construction and target-promotion pathway materially increases the rate at which sparse INT8 faults realize declared semantic substitutions under the decode-time gate cascade.

D.2 Acceptance-Mode Configuration Reference: Decode-Only vs Hybrid

This is not a controlled ablation. The two configurations compared here differ on multiple axes in addition to acceptance mode: the target-strength γ , `smash_decode_trials`, the ranking-flag set (`smash_ent_logit_impact_ranking`, `smash_neg_logit_impact_ranking`), the negation-shrinking-target flag, and the negation bundle size. Any observed difference in strict rate therefore reflects a joint configuration choice rather than the isolated effect of switching acceptance mode. We include the comparison only to document the acceptance-mode settings that produced the main-text strict-rate numbers and to make replication of those numbers unambiguous.

This subsection accompanies the reference in Section 5. We compare the strict decode-only reference configuration against the hybrid acceptance strategy used in the main paper. The comparison separates the report-time success criterion from the search-time acceptance rule.

D.2.1 Decode-only acceptance

Decode-only acceptance commits a candidate only when the full gate cascade **G1–G4** holds on the decoded transcript after the candidate is applied. This is the strictest search-time acceptance rule: every committed update is already a verified semantic fault under the threat model, with no surrogate-driven intermediate moves.

The decode-only configuration retained here corresponds to the family-A configuration developed first during method tuning. It uses `Whisper-small.en`, the `whisper_decoder_last2_plus_lm_head` preset, decode-only acceptance, `smash_decode_trials=6`, target strength $\gamma = 20$, and the auxiliary settings `smash_neg_shrinking_target=1`, `smash_ent_logit_impact_ranking=1`, `smash_neg_logit_impact_ranking=1`, and `smash_neg_bundle_size=3`. We retain this configuration as a strict-acceptance reference, not as the canonical main-paper configuration.

D.2.2 Reference comparison

Table 8 reports strict-success count on `Whisper-small.en` $\times$ LACC across the budget sweep. Hybrid acceptance is the canonical configuration of the main paper: $\gamma = 10$, `smash_decode_trials=2`, default ranking, and default bundle settings. Decode-only uses the family-A configuration described above.

Table 8: **Strict-success counts at $N=100$.** At $B=20$ the hybrid configuration recovers roughly $13\times$ more strict successes than the decode-only reference; note that the reference also differs in γ , `smash_decode_trials`, ranking flags, and negation bundle settings (see §D.2, “Decode-only acceptance”), so the gap should be read as a strict-acceptance reference rather than an isolated acceptance-rule ablation.

Acceptance	$B=1$	$B=2$	$B=3$	$B=4$	$B=5$	$B=10$	$B=20$	$B=40$
Decode-only reference	0	2	2	2	2	2	2	2
Hybrid canonical	0	14	18	20	22	25	26	26

The decode-only reference plateaus at 2 strict successes from $B=2$ onward; the hybrid configuration crosses double digits at $B=2$ and saturates at 26 successes by $B=20$.

D.2.3 Interpretation: search-time bridge versus report-time success

The gap is not a relaxation of the reported success criterion. Both configurations report only certificates that satisfy the full gate cascade **G1–G4** at decode time on the final committed flip set. The difference is the search-time rule used to reach that final state.

Decode-only acceptance rejects moves that improve the proposal objective but have not yet caused the decoded transcript to realize the declared target. In an autoregressive decoder, small parameter perturbations often shift internal scores without crossing the decode boundary. Hybrid acceptance permits the search to retain such surrogate-improving moves while an intermediate WER cap is held, allowing several small parameter changes to accumulate before the decoded output transitions. A sample is still counted only when the final decoded transcript satisfies **G1–G4**.

Table 9: **Full strict-success budget sweep.** Strict counts for canonical SMASH-hybrid on each model–dataset cell. Rates should be computed as $\text{strict}/|T|$. The $B=100$ column is a LACC sidecar only and is not part of the main grid.

Model	Corpus	N	$ T $	$B=1$	$B=2$	$B=3$	$B=4$	$B=5$	$B=10$	$B=20$	$B=40$	$B=100$
WS	LACC	100	84	0	14	18	20	22	25	26	26	26
WS	MultiMed	100	31	0	1	2	3	3	5	5	5	–
WS	LibriSpeech	100	31	0	3	3	3	5	7	7	7	–
WLv3	LACC	100	82	0	1	1	1	2	7	21	25	25
WLv3	MultiMed	100	39	0	0	0	0	0	0	2	4	–
WLv3	LibriSpeech	100	30	0	0	0	0	0	1	1	2	–
S-M4T	LACC	100	82	0	0	0	0	0	0	6	12	12
S-M4T	MultiMed	100	42	2	2	2	2	2	2	2	3	–
S-M4T	LibriSpeech	100	31	0	0	0	0	0	0	0	1	–

Because the decode-only reference also differs in several tuning knobs, its absolute count is a strict-acceptance reference rather than a perfectly isolated ablation of acceptance strategy. The qualitative conclusion is nevertheless clear: requiring every intermediate committed state to satisfy **G4** makes the search much less efficient, while hybrid acceptance recovers the reachable set without relaxing the final reporting metric.

E Full Results and Uncertainty

This appendix consolidates the full per-cell results and the uncertainty/sensitivity evidence for the main SMASH-hybrid rates. §E.1 reports the per-cell budget sweep used to compute the headline numbers in Section 6. §E.2 reports Wilson 95% confidence intervals for the rate estimates. §E.3 sweeps the bundle size, transcript-proximity threshold, and target strength to verify that the canonical configuration lies in a stable regime rather than at a knife-edge hyperparameter setting.

E.1 Full Per-Cell Budget Sweep

This appendix provides the full per-cell budget sweep underlying Section 6. The main text reports the compact cross-model/cross-corpus targetability table at $B=20$ and the saturation curves; here we report the raw strict-success counts across budgets for every model–corpus cell.

Unless otherwise stated, all rows use the canonical SMASH-hybrid configuration from Section 5: deterministic decoding, seed 42, $N=100$ samples per cell, automatic target construction over $\{\text{num}, \text{ent}, \text{neg}\}$ (numeric/scalar outcomes reported in this paper; entity and negation construction is a legacy all-pivot diagnostic), bundle size 1, $K=3$, $\tau_{\text{wer}}=0.20$, and sign-bit flips excluded. Strict counts are shown as raw counts; strict rates in the main text are computed as $\text{strict}/|T|$, where $|T|$ is the target-constructible denominator.

E.1.1 Strict-success counts across budgets

Table 9 reports strict-success counts for each model–corpus cell across the main budget sweep

$$B \in \{1, 2, 3, 4, 5, 10, 20, 40\}.$$

For LACC, we additionally include the $B=100$ sidecar used in Section 6. The sidecar is not part of the main grid; it is included only to test whether the synthetic reachable set grows beyond the high end of the main sweep.

The main pattern is saturation. Strict success is non-decreasing in B , as expected, but most cells plateau by $B=20$ – 40 . On LACC, the $B=100$ sidecar produces no additional strict successes beyond the high-budget plateau on any backbone. This supports the interpretation in Section 6: within the canonical perturbation class and acceptance cascade, the measured reachable set is structurally bounded rather than simply under-budgeted.

Table 10: **Movement decomposition at $B=20$ over all N samples.** No-effect ($WER=0$), local ($0 < WER \leq 0.20$), and drift ($WER > 0.20$) are over N and partition each row. Partial uses the overlapping definition (target constructed, $G1 \wedge G2 \wedge G3$ pass, source pivot span count drops in the adversarial transcript; no strict-fail guard. Partial overlaps with strict by construction; partial $\geq$ strict where source displacement accompanies target realization. Values are verified per-sample logs.

Model	Corpus	N	$ T $	No eff.	Local	Drift	Partial	Strict
WS	LACC	100	84	59	40	1	25	26
WS	MultiMed	100	31	84	14	2	8	5
WS	LibriSpeech	100	31	81	18	1	7	7
WLv3	LACC	100	82	70	29	1	25	21
WLv3	MultiMed	100	39	73	27	0	7	2
WLv3	LibriSpeech	100	30	84	15	1	3	1
S-M4T	LACC	100	82	52	42	6	29	6
S-M4T	MultiMed	100	42	64	27	9	8	2
S-M4T	LibriSpeech	100	31	73	22	5	2	0

E.1.2 Movement decomposition at the canonical budget

Table 10 reports the post-hoc movement decomposition at the canonical analysis budget $B=20$. No-effect, local-movement, and high-WER drift categories partition all $N=100$ samples in each cell. Source-disrupted partials and strict successes are reported as additional target-conditioned raw counts on the target-constructible subset $|T|$. Following the main-paper definition (Section 5), partial is computed without a strict-fail guard: it counts every sample where a target was constructed, $G1 \wedge G2 \wedge G3$ pass, and the source pivot span appears fewer times in the adversarial transcript than in the baseline. Partial therefore overlaps with strict by construction, and partial ≥ 0 is expected to be at least the strict count whenever the search displaces the source span on accepted samples.

This decomposition separates two phenomena that are conflated by aggregate transcript-change metrics. Some cells produce many local edits but few strict target hits; others produce a smaller number of edits but a cleaner strict-success profile. Source-disrupted partials indicate that the selected source span was affected without a strict target realization; they are diagnostic of source-side movement, not an alternative success metric.

E.1.3 Relationship to main-text rates

The strict counts in Table 9 should be read together with the target-constructible denominator $|T|$. The main paper reports StrictRate as strict/ $|T|$, not strict/ N . We include N and $|T|$ in the appendix tables so that readers can recompute every rate reported in Section 6 and verify that cross-corpus differences are not caused by a hidden denominator shift.

For example, LACC has the largest $|T|$ because it is constructed to contain dense meaning-critical spans. MultiMed and LibriSpeech contain fewer target-constructible opportunities, so their raw strict counts are lower partly because the target-construction ceiling is lower. This is why the main text distinguishes target coverage from search efficacy.

E.1.4 Real-speech budget evidence is matched up to $B=40$; $B=100$ is LACC-only

The matched $B=20 \rightarrow B=40$ numeric-pivot sweep is complete on all six real-speech cells in Table 9 (WS, WLv3, S-M4T $\times$ MultiMed, LibriSpeech). The $B=100$ sidecar control was run on LACC only; we therefore phrase the plateau finding in Section 6 as a controlled-corpus claim and do not extend it to real speech. Cells not run are reported as *not run* (rendered as “–”) and we do not impute zero values for missing cells.

E.2 Confidence Intervals

This appendix reports Wilson score 95% confidence intervals for the main rate estimates. We use Wilson intervals rather than normal approximations because several cells have small target-constructible denominators or zero observed successes. For target-conditioned SMASH rates,

Table 11: **Numeric/scalar targetability across models $\times$ datasets at $B=20$ SMASH-hybrid.** Per-cell denominator is $|T_{\text{num}}|$ (numeric-target-constructible samples in the cell), drawn from the `Target_built` table at $B=20$, $N=100$, seed 42. Wilson 95% CIs computed from (strict, $|T_{\text{num}}|$). The MultiMed $\times$ S-M4T cell (0/25) and the LibriSpeech $\times$ {WLv3, S-M4T} cells are bona-fide zeros. LibriSpeech denominators are small ($|T_{\text{num}}| \in \{8, 9, 10\}$); the wide CIs reflect that small-sample uncertainty rather than imprecision of the underlying measurement.

Corpus	Model	$ T_{\text{num}} $	Strict	Partial	Rate	95% CI
LACC	WS	45	24	24	0.533	[0.391, 0.671]
LACC	WLv3	44	19	21	0.432	[0.297, 0.578]
LACC	S-M4T	34	5	20	0.147	[0.065, 0.302]
MultiMed	WS	20	4	4	0.200	[0.081, 0.416]
MultiMed	WLv3	26	2	5	0.077	[0.021, 0.241]
MultiMed	S-M4T	25	0	4	0.000	[0.000, 0.133]
LibriSpeech	WS	9	5	4	0.556	[0.267, 0.811]
LibriSpeech	WLv3	8	0	0	0.000	[0.000, 0.324]
LibriSpeech	S-M4T	10	0	0	0.000	[0.000, 0.278]

Table 12: **Wilson 95% CIs for cross-cell strict-success rate at $B=20$.** Rates are $\# \text{strict} / |T|$ over the target-constructible subset. Wilson intervals are robust to the small denominators on real-world data. (Pooled-pivot view; the numeric-only matched view is in Table 11.)

Corpus	Model	$ T $	Strict	Rate	Wilson 95% CI
LACC	WS	84	26	0.310	[0.221, 0.415]
LACC	WLv3	82	21	0.256	[0.174, 0.360]
LACC	S-M4T	82	6	0.073	[0.034, 0.151]
MultiMed	WS	31	5	0.161	[0.071, 0.326]
MultiMed	WLv3	39	2	0.051	[0.014, 0.169]
MultiMed	S-M4T	42	2	0.048	[0.013, 0.158]
LibriSpeech	WS	31	7	0.226	[0.114, 0.398]
LibriSpeech	WLv3	30	1	0.033	[0.006, 0.167]
LibriSpeech	S-M4T	31	0	0.000	[0.000, 0.110]

intervals are computed over $|T|$. For cross-method absolute event-rate comparisons, intervals are computed over the evaluated sample count N .

The confidence intervals support the main separation claims. On $\text{WS} \times \text{LACC}$, SMASH-hybrid produces a nonzero strict-event rate over N with Wilson 95% CI [0.184, 0.354], while every untar-geted baseline has upper bound at most 0.037 in the matched cross-method table. The validation-set separation is also large: on the $n=105$ shared validation set (Section 6, Appendix H), SMASH-hybrid has Wilson clean-rate CI [0.665, 0.939], while non-SMASH baselines have upper bounds at most 0.390. Cross-dataset degradation is visible in the target-conditioned intervals, especially for WLv3 and S-M4T real-speech cells. We avoid pairwise non-overlap claims because the S-M4T synthetic interval [0.034, 0.151] overlaps several real-speech intervals, and we report intervals so readers can assess specific comparisons directly.

E.3 Hyperparameter Sensitivity

This appendix accompanies the references in Sections 4 and 5. We sweep three search and acceptance hyperparameters on Whisper-small.en $\times$ LACC at $B=20$: bundle size, transcript-proximity threshold, and target strength. The goal is to check whether the canonical configuration lies in a stable regime rather than at a knife-edge setting.

All sweep cells hold the non-swept knobs at family-A tuning ($\gamma=20$ where not swept, LIR on with `smash_ent_logit_impact_ranking=1` and `smash_neg_logit_impact_ranking=1`, `smash_neg_shrinking_target=1`, and `smash_neg_bundle_size=3`). The main-paper canonical configuration uses $\gamma=10$ and LIR off but the same swept-knob values (bundle size 1, $\tau_{\text{wer}} = 0.20$);

Table 13: **Wilson 95% CIs along the SMASH-hybrid budget sweep on LACC.** Rates are $\# \text{strict} / |T|$ for each model. The $B=100$ row is a sidecar; $|T|$ is fixed per model. Saturation $B \rightarrow 100$ is verified: the WS, WLv3, and S-M4T strict CIs do not move past their $B=40$ values.

B	WS ($ T =84$)		WLv3 ($ T =82$)		S-M4T ($ T =82$)	
	Rate	95% CI	Rate	95% CI	Rate	95% CI
1	0.000	[0.000, 0.044]	0.000	[0.000, 0.045]	0.000	[0.000, 0.045]
2	0.167	[0.102, 0.261]	0.012	[0.002, 0.066]	0.000	[0.000, 0.045]
3	0.214	[0.140, 0.313]	0.012	[0.002, 0.066]	0.000	[0.000, 0.045]
4	0.238	[0.160, 0.339]	0.012	[0.002, 0.066]	0.000	[0.000, 0.045]
5	0.262	[0.180, 0.365]	0.024	[0.007, 0.085]	0.000	[0.000, 0.045]
10	0.298	[0.210, 0.402]	0.085	[0.042, 0.166]	0.000	[0.000, 0.045]
20	0.310	[0.221, 0.415]	0.256	[0.174, 0.360]	0.073	[0.034, 0.151]
40	0.310	[0.221, 0.415]	0.305	[0.216, 0.411]	0.146	[0.086, 0.239]
100	0.310	[0.221, 0.415]	0.305	[0.216, 0.411]	0.146	[0.086, 0.239]

Table 14: **Wilson 95% CIs for the cross-method comparison on WS $\times$ LACC.** Rates in this table are absolute strict-event rates over the evaluated sample set N , used for cross-method separation. Some baseline rows aggregate multiple evaluated slices; reported N is the pooled evaluated count for that method, with per-cell breakdowns reported in Appendix F. Target-conditioned SMASH StrictRate values over $|T|$ are reported in Table 12.

Method	Targeted?	B	N	Strict / N	Wilson 95% CI
SMASH (hybrid)	yes	20	100	26/100 = 0.260	[0.184, 0.354]
SMASH (hybrid)	yes	40	100	26/100 = 0.260	[0.184, 0.354]
PBS-CE	no	20	100	0/100 = 0.000	[0.000, 0.037]
PBS-CE	no	40	100	0/100 = 0.000	[0.000, 0.037]
PBS-CE-gated	no	20	180	0/180 = 0.000	[0.000, 0.021]
URBF	no	20	100	0/100 = 0.000	[0.000, 0.037]
TGRF-CE	no	20	100	0/100 = 0.000	[0.000, 0.037]
TGRF-MSB	no	20	100	0/100 = 0.000	[0.000, 0.037]
TGRF-Uniform	no	20	100	0/100 = 0.000	[0.000, 0.037]
SilentStriker-native	yes	40	200	0/200 = 0.000	[0.000, 0.019]
SilentStriker-CE	yes	40	100	0/100 = 0.000	[0.000, 0.037]
SilentStriker-CE-gated	yes	40	200	0/200 = 0.000	[0.000, 0.019]

these sweeps therefore probe the stability of the swept knobs, not a denominator-matched main-paper ablation.

E.3.1 Bundle size

Bundle size is the number of high-ranked candidate flips that SMASH evaluates jointly per search step (Section 4). Larger bundles cross the autoregressive decoder boundary more aggressively but can commit more flips per step. Table 15 reports strict success and realized cost across bundle sizes 1, 2, and 3 with all other settings held fixed.

Table 15: **Bundle-size sensitivity on WS $\times$ LACC at $B=20$.** Strict counts are reported for the matched sweep. The $|F|$ mean is computed over strict-success samples only. Larger bundles slightly increase strict count but also increase realized perturbation cost.

Bundle size	Strict	$ F $ Mean
1	25	3.2
2	25	3.9
3	29	4.8

Bundle sizes 1 and 2 produce the same strict count in this sweep. Bundle size 3 produces four additional strict successes at higher realized cost. We adopt bundle size 1 as canonical because it gives more parsimonious certificates while preserving the main reachable set.

E.3.2 Plausibility threshold τ_{wer}

The transcript-proximity component of gate **G3** requires $\text{WER}(\hat{y}_F, y_0) \leq \tau_{\text{wer}}$. The canonical setting is $\tau_{\text{wer}} = 0.20$. Table 16 sweeps τ_{wer} on WS and WLv3 at $B=20$.

Table 16: **Transcript-proximity threshold sensitivity at $B=20$** . Counts report strict successes as τ_{wer} is varied. The final column disables the plausibility gate **G3** (-mislead_require_plausible 0) and should be read as an upper-bound diagnostic rather than a reported operating point.

Model	$\tau=0.10$	$\tau=0.15$	$\tau=0.20$	$\tau=0.25$	$\tau=0.30$	G3 off
WS	17	21	25	26	27	29
WLv3	17	22	22	22	22	22

WS strict success rises as the threshold relaxes from 0.10 to 0.20, then increases only modestly beyond that point. WLv3 reaches its plateau by $\tau_{\text{wer}} = 0.15$. The canonical $\tau_{\text{wer}} = 0.20$ lies in a stable regime for both backbones.

E.3.3 Target strength γ

The target-promotion term in $\mathcal{L}_{\text{SMASH}}$ is weighted by γ . Table 17 sweeps γ on WS $\times$ LACC at $B=20$.

Table 17: **Target-strength sensitivity on WS $\times$ LACC at $B=20$** . Strict success is approximately constant across the tested range.

	$\gamma = 5$	$\gamma = 10$	$\gamma = 15$	$\gamma = 20$	$\gamma = 30$
Strict	24	25	25	25	24

Strict success is approximately flat across $\gamma \in \{5, 10, 15, 20, 30\}$. The canonical $\gamma = 10$ is adopted because it lies in the stable part of the sweep and matches the default configuration.

E.3.4 Summary

Across bundle size, transcript-proximity threshold, and target strength, the canonical configuration lies within a robust regime. Strict reachability under the main SMASH configuration is therefore not an artifact of a finely tuned hyperparameter setting.

F Cross-Method Comparison

This subsection expands the cross-method comparison summarized in Section 6. The purpose of the comparison is not to show that SMASH maximizes transcript movement. It is to distinguish two different measurement events: untargeted transcript movement versus declared targeted semantic substitution. Baseline bit-flip methods can change transcripts, but under the same post-hoc target-realization gates they do not realize the declared source $\rightarrow$ target semantic edit.

F.1 Baseline adaptation protocol

We adapt prior bit-flip baselines to the same quantized seq2seq ASR setting by holding fixed the model, dataset, sample manifest, quantization scheme, parameter preset, seed, decoding configuration, and target-construction pipeline whenever the corresponding run is available. **PBS-CE** adapts Progressive Bit Search by ranking INT8 bit flips using the gradient of teacher-forced cross-entropy on the quantized baseline transcript y_0 ; it is untargeted because it maximizes transcript loss rather than promoting the declared semantic target. **PBS-CE-gated** uses the same proposal rule but only commits decoded outputs that pass non-target readability and proximity gates. **SilentStriker-native** is configured as a targeted attack in its native form (module-level sign-bit search); on the WS $\times$ LACC matrix the auto-target builder did not engage on any sample, so the reported runs executed under the family’s untargeted CE-fallback objective, as disclosed in the baseline-results discussion. We retain these rows as evidence about transcript movement under that fallback, not as successful SilentStriker

Table 18: **Per-method adaptation summary for the cross-method comparison.** Each method uses its native objective and bit-position policy. We keep the model, dataset, sample manifest, quantization scheme, parameter preset, seed, decoding configuration, and target-construction pipeline fixed whenever the corresponding run is available. **Bit policy** reports candidate INT8 bit positions: 0..6 excludes the sign bit, 0..7 includes it, and 7-only means sign-bit only.

Method	Search objective	Acceptance rule	Bit policy	Targeted?
SMASH-hybrid	Span-conditioned objective with target term	Hybrid proposal plus G1–G4	0..6	Yes
SMASH-decode	Span-conditioned objective with target term	Decode-only G1–G4	0..6	Yes
PBS-CE	Teacher-forced CE on y_0 ; maximized	None	0..7	No
PBS-CE-gated	Teacher-forced CE on y_0 ; maximized	Readability and proximity gates	0..7	No
SilentStriker-native	Native module-level sign-bit search; CE fallback on $WS \times LACC$	Gate cascade	7-only	Native targeted; fallback here
SilentStriker-CE	Teacher-forced CE on y_0 ; maximized	None	0..6	No
SilentStriker-CE-gated	Teacher-forced CE on y_0 ; maximized	Readability and proximity gates	0..6	No
URBF	Uniform random flip locations	None	0..7	No
TGRF-CE	Top- K gradient locations with CE acceptance	Gated	0..7	No
TGRF-MSB	Top- K gradient locations with MSB-only flips	Gated	7-only	No
TGRF-Uniform	Top- K gradient locations with uniform bit choice	Gated	0..7	No

target construction. **SilentStriker-CE** replaces the native objective with the same teacher-forced CE objective used for the PBS adaptations; **SilentStriker-CE-gated** adds the same decoded-output gates. **URBF** samples flip locations uniformly from the eligible parameter pool. The **TGRF** family draws candidate locations from the top- K coordinates by gradient magnitude and varies the bit policy across three variants: `tgrf_ce` uses CE-driven acceptance, `tgrf_msb` is most-significant-bit only, and `tgrf_uniform` samples uniformly over eligible bit positions. None of these non-SMASH baselines optimizes the declared source→target replacement; they are nevertheless evaluated for strict success under the same **G1–G4** cascade, so a baseline receives credit if it happens to realize the declared semantic target while remaining readable and close to y_0 .

Bit-position policy is per-method native, not matched: Each method uses the bit-position policy that follows from its native paper definition rather than a forced common policy. The canonical SMASH configuration uses bit positions 0..6, excluding the sign bit, because INT8 sign-bit flips can shift a single dequantized weight by up to $128|s_i|$ and are uniquely catastrophic; `smash_allow_sign_bit=0`. The PBS family follows the original PBS specification and allows bit positions 0..7, including the sign bit; `pbs_allow_sign_bit=1`. The random-search family, including URBF and TGRF, likewise allows 0..7; `rand_allow_sign_bit=1`. The `tgrf_msb` variant is a deliberate MSB-only ablation. SilentStriker-native is sign-bit-only by paper design, since its module-level routine flips bit 7. SilentStriker-CE and SilentStriker-CE-gated share the canonical SMASH bit positions 0..6 because they reuse the same masked-break code path. The practical consequence is that SMASH draws its sparse certificate from a strictly smaller per-scalar candidate pool, 7 bits per location, than PBS/URBF/TGRF, 8 bits per location, and yet SMASH produces strict target realizations while those baselines do not. The comparison is therefore conservative with respect to perturbation freedom.

Budget and compute fairness: All methods are reported under their declared bit-budget and step-budget envelopes; we do not give SMASH a larger bit budget than the comparison baselines.

Table 19: **Audit-verified cross-method movement decomposition.** Counts are over N on $WS \times LACC$. No effect means $WER = 0$, local edit means $0 < WER \leq 0.20$, and high-WER drift means $WER > 0.20$. Strict target-hit is the only column that requires realization of the declared source→target semantic edit.

Method	Targeted?	B	N	No eff.	Local	Drift	Strict
SMASH (hybrid)	yes	20	100	59 (59.0%)	40 (40.0%)	1 (1.0%)	26 (26.0%)
SMASH (decode)	yes	20	100	84 (84.0%)	16 (16.0%)	0 (0.0%)	2 (2.0%)
PBS-CE	no	20	100	20 (20.0%)	0 (0.0%)	80 (80.0%)	0
PBS-CE-gated	no	20	180	92 (51.1%)	88 (48.9%)	0 (0.0%)	0
URBF	no	20	100	100 (100%)	0 (0.0%)	0 (0.0%)	0
TGRF-CE	no	20	100	61 (61.0%)	39 (39.0%)	0 (0.0%)	0
TGRF-MSB	no	20	100	98 (98.0%)	2 (2.0%)	0 (0.0%)	0
TGRF-Uniform	no	20	100	100 (100%)	0 (0.0%)	0 (0.0%)	0
PBS-CE	no	40	100	20 (20.0%)	0 (0.0%)	80 (80.0%)	0
SilentStriker-native	yes	40	200	138 (69.0%)	7 (3.5%)	55 (27.5%)	0
SilentStriker-CE	yes	40	100	10 (10.0%)	6 (6.0%)	84 (84.0%)	0
SilentStriker-CE-gated	yes	40	200	188 (94.0%)	12 (6.0%)	0 (0.0%)	0

Differences in reported strict counts therefore reflect proposal objective, target conditioning, and acceptance rule rather than a larger allowed perturbation budget for SMASH.

Decode determinism: Decoding is deterministic under fixed seed and the canonical greedy/beam-1 configuration from Section 5. Accepted transcript differences across methods are therefore attributable to parameter perturbations rather than sampling stochasticity.

F.2 Movement decomposition

Table 19 reports the audit-verified movement decomposition. We decompose each method’s outputs into no effect, local edit, and high-WER drift. No effect means $WER = 0$ relative to the quantized baseline, local edit means $0 < WER \leq 0.20$, and high-WER drift means $WER > 0.20$. Strict target-hit is reported separately because it requires realization of the declared source→target semantic edit.

The movement decomposition explains the baseline failure modes. PBS-CE and SilentStriker-CE move many transcripts, but much of that movement is high-WER drift. Gated variants suppress high-WER drift and convert many outputs into local edits, but still produce no strict target hits because the gate has no notion of which semantic substitution should be realized. Random and top-gradient random baselines are mostly inert or produce local movement without target realization.

F.3 Movement-conditional view

Table 20 conditions on the subset of samples whose transcript actually changed. This removes the no-effect bucket and asks what kind of movement each method produces when it moves the transcript at all. The strict/moved column is a separate diagnostic and can overlap with local movement: strict successes usually also satisfy the local-WER condition.

This view is useful because it prevents a misleading interpretation of no-effect-heavy methods. Among moved samples, SMASH-hybrid converts 63.4% of decode changes into strict targeted substitutions, while every non-SMASH baseline converts 0%. Untargeted CE methods often move transcripts, but the movement is usually high-WER drift. Gated CE methods keep movement local, but the movement remains off-target.

F.4 Interpretation

The comparison should be read as an event-alignment result rather than a horse race. Untargeted baselines optimize transcript movement, cross-entropy increase, or randomized bit perturbation. SMASH optimizes a stricter event: a declared semantic pivot must change into a declared target while the transcript remains readable and plausible. Under this event definition, raw movement is

Table 20: **Movement-conditional decomposition.** Restricted to samples with WER > 0 . %Strict/Moved is the fraction of moved samples that hit the strict target gate. This strict column is a separate diagnostic and can overlap with %Local/Moved because strict successes typically satisfy the local-WER condition.

Method	B	N	Moved	%Strict	%Local	%Drift
SMASH (hybrid)	20	100	41	63.4%	97.6%	2.4%
SMASH (decode)	20	100	17	11.8%	82.4%	17.6%
PBS-CE	20	100	80	0.0%	0.0%	100.0%
PBS-CE-gated	20	180	88	0.0%	100.0%	0.0%
URBF	20	100	0	—	—	—
TGRF-CE	20	100	39	0.0%	100.0%	0.0%
TGRF-MSB	20	100	2	0.0%	100.0%	0.0%
TGRF-Uniform	20	100	0	—	—	—
PBS-CE	40	100	80	0.0%	0.0%	100.0%
SilentStriker-native	40	200	62	0.0%	11.3%	88.7%
SilentStriker-CE	40	100	90	0.0%	6.7%	93.3%
SilentStriker-CE-gated	40	200	12	0.0%	100.0%	0.0%

insufficient. A robustness audit that measures only whether transcripts change would miss the failure mode SMASH exposes: sparse, plausible-looking substitutions on meaning-critical spans.

F.5 Matched qualitative comparison

Table 21 controls for the audio: five WS $\times$ LACC samples are run through five methods at $B=20$, with the declared source $\rightarrow$ target edit held fixed within each row group. SMASH-hybrid realizes the declared target on every retained sample; the four non-SMASH baselines do not land the declared target, producing instead no movement, case-only edits, spelling changes, or off-target local edits. The full per-sample gallery and the rejected-candidate audit table are not reproduced in the paper PDF.

G Threat-Model Locality Checks

This appendix reports necessary-condition checks on the locality of the bit-flip sets that realize SMASH targets under the canonical configuration. The intent is to characterize whether successful flip sets concentrate in a manner consistent with the abstract threat model assumed in the main text. We do not claim — and these checks do not demonstrate — end-to-end feasibility of a physical fault-injection exploit against any specific deployed system. The constraints below operate over positions within targeted weight tensors; they do not simulate DRAM row mapping, refresh timing, cache coherence, or a full physical attack path. Results should be read as feasibility-constraint evidence for the threat model as stated, and nothing further.

The locality experiments were run in three phases. Phase 1 is a decode-only sanity check with `max_steps=1`. Phase 1b is the main hybrid locality sweep on Whisper-small.en $\times$ LACC at $B=20$. Phase 2 spot-checks the two burst-style locality modes across additional model-dataset cells.

Locality-mode definitions: The seven constrained modes restrict the candidate flip set within targeted weight tensors as follows. `page_local`: candidates restricted to a single 4 KiB page of contiguous bytes per attack step. `rowhammer_row` [14]: candidates restricted to a single DRAM-row-analogue window. `rowpress` [19]: row-restricted and additionally concentrated within a sub-row burst. `half_double` [15]: candidates allowed across the chosen row plus its row- ± 2 neighbours. `cacheline_burst`: candidates restricted to a single 64 B cacheline. `blacksmith_burst` [12]: candidates allowed across multiple non-contiguous rows in a burst pattern. `collateral_noise`: candidates position-unrestricted but with random no-op flips injected to model interference. The unconstrained none arm is the canonical SMASH-hybrid setting.

Denominator note: All locality results in this appendix use the matched numeric-pivot denominator $|T_{\text{num}}|$ from the canonical budget-sweep cell ($|T_{\text{num}}|=45$ for WS $\times$ LACC, 44 for WLv3 $\times$ LACC, 34

Table 21: **Matched WS×LACC samples at $B=20$: same audio, same declared target, five methods.** SMASH-hybrid realizes the declared substitution on every row (target span in **bold**); the four non-SMASH baselines either do not move the transcript, produce off-target casing/spelling edits (in *italics*), or collapse into local drift, but none lands the declared target. “–” indicates the adversarial transcript is byte-identical to the quantized baseline. WER is computed against the quantized baseline transcript; rows showing a differing transcript at WER= 0.00 are case- or punctuation-only changes that the metric normalizes away.

Sample (src → tgt)	Method	WER	Adversarial transcript
num_002 (38 → 48)	SMASH-hybrid	0.10	The thermometer read 48 degrees celsius during the afternoon check.
	PBS-CE-gated	0.00	<i>rhe</i> thermometer read 38 degrees celsius during the afternoon check.
	SilentStriker-native	0.00	–
	TGRF-CE	0.00	The thermometer read 38 degrees celsius during the afternoon check.
	URBF	0.00	–
num_009 (\$45 → \$55)	SMASH-hybrid	0.11	We spent \$55 on groceries at the corner market.
	PBS-CE-gated	0.00	We spent \$45 on groceries at the <i>Corner Market</i> .
	SilentStriker-native	0.00	–
	TGRF-CE	0.00	We spent \$45 on groceries at the <i>Corner Market</i> .
	URBF	0.00	–
num_020 (20 → 30)	SMASH-hybrid	0.10	The vaccine was stored at minus 30 degrees for stability.
	PBS-CE-gated	0.10	The vaccine was stored at minus 20 degrees for <i>Stabilityibil-</i> <i>ity</i> .
	SilentStriker-native	0.00	–
	TGRF-CE	0.00	–
	URBF	0.00	–
mix_007 (500 → 600)	SMASH-hybrid	0.09	Parboarview Studios didn’t ship 600 grams of resin with the order.
	PBS-CE-gated	0.00	–
	SilentStriker-native	0.00	–
	TGRF-CE	0.09	<i>Parboreview</i> Studios didn’t ship 500 grams of resin with the order.
	URBF	0.00	–
mix_022 (30 → 40)	SMASH-hybrid	0.09	Camden Lee didn’t wait 40 seconds before pressing the reset button.
	PBS-CE-gated	0.00	–
	SilentStriker-native	0.00	–
	TGRF-CE	0.00	Camden <i>lee</i> didn’t wait 30 seconds before pressing the reset button.
	URBF	0.00	–

for S-M4T×LACC, 4 for WS×LibriSpeech), so every cell here directly verifies the headline numbers reported in Section 6. Partial follows the strict-inclusive convention used throughout the paper: a sample counts as partial when a target was constructed, **G1/G2/G3** pass, and the source pivot span count drops in the adversarial transcript, regardless of whether strict **G4** also holds. Partial therefore overlaps with strict by construction (Section 6). The locality pipeline runs use the same codebase and per-cell JSONL records as the main experiments.

Table 22: **Locality-realism sweep, Phase 1b (hybrid).** WS×LACC at $B=20$, $N=100$, $|T_{\text{num}}|=45$, seed 42. Numeric-pivot subset only; partial follows the strict-inclusive convention (target constructed, **G1**∧**G2**∧**G3** pass, source span count drops; overlaps with strict). Five of seven constrained modes preserve the unconstrained strict count 24/45; rowpress is the main penalty case. Strict counts in this table are target-realized (**G1**–**G4**); clean-substitution analysis additionally requires source disappearance or material source change (Section 6, Section 3).

Locality mode	Strict	Partial	G1	No eff.	Local	Drift
none (baseline)	24	23	28	17	27	1
page_local	24	22	26	19	26	0
rowhammer_row	24	22	26	19	26	0
rowpress	15	12	18	28	17	0
half_double	22	24	24	21	23	1
cacheline_burst	24	25	28	17	27	1
blacksmith_burst	24	23	28	17	27	1
collateral_noise	24	22	27	18	27	0

G.1 Phase 1: decode-only sanity check

A Phase 1 reference under decode-only acceptance with `max_steps=1` on Whisper-small.en×LACC at $B=20$ ($N=100$, seed 42) reaches no strict success in any locality mode. Movement counters are identical across modes except for `blacksmith_burst`, which reduces decode-change activity from 19 to 12 samples. This phase confirms that the locality machinery itself does not create strict successes; the targeted locality diagnostic is the hybrid Phase 1b sweep below.

G.2 Phase 1b: hybrid locality sweep on WS×LACC

Table 22 reports the main locality sweep using hybrid acceptance, `max_steps=30`, and $B=20$.

The unconstrained `none` arm reaches 24/45 strict successes, exactly matching the canonical budget-sweep cell in Table 9. Five of the seven constrained modes preserve this count exactly: `page_local`, `rowhammer_row`, `cacheline_burst`, `blacksmith_burst`, and `collateral_noise`. `half_double` drops to 22/45, and `rowpress` is the main penalty case at 15/45. High-WER drift remains at most 1 sample in every mode, indicating that the plausibility gate continues to control transcript degradation under locality constraints.

The reason WS is largely insensitive to locality on LACC is quantitative: WS strict-accepted certificates have median $|F|=2$ bit flips (Figure 3), so the committed flip footprint fits trivially inside any of the constrained windows tested here. Only `rowpress`, which adds a sub-row burst restriction on top of the row constraint, cuts deeply into the search space the WS attack uses.

G.3 Phase 2: cross-model burst spot checks

Table 23 reports cross-model spot checks for the two burst-style locality modes that most likely trigger preset-dependent placement sensitivity.

The structural finding lives on the harder backbones. **WLv3×LACC swings from 4/44 strict under `cacheline_burst` to 17/44 under `blacksmith_burst`, a 4× preset-dependent gap on the same corpus.** S-M4T×LACC produces 0/34 strict successes under either burst mode, despite 9–17 decode changes and 9–14 partials. WS×LibriSpeech reaches 1/4 under both modes; the cell is too small to draw conclusions beyond the WLv3 finding. The unconstrained reference values from Table 9 (19/44 WLv3, 5/34 S-M4T) bracket the burst results: `blacksmith` preserves nearly all of WLv3’s reach (17/19), `cacheline` preserves only one fifth (4/19), and S-M4T loses both bursts entirely.

Why `cacheline_burst` collapses WLv3 reach: Cacheline-burst restricts every accepted flip to a single 64 B cacheline. WLv3 strict-accepted certificates have median $|F|=11$ bit flips on LACC, and the targeted WDL2 weights are densely packed enough that 11 flips cannot reliably be co-located in a single cacheline. `blacksmith_burst` relaxes the constraint to multiple non-contiguous bursts and recovers 17/19 of the unconstrained reach. WS does not show this collapse because its strict accepts have median $|F|=2$, which fits in any reasonable cacheline window. S-M4T’s failure mode is different

Table 23: **Locality-realism cross-model spot checks, Phase 2.** Hybrid acceptance, max_steps=30, B=20. Numeric-pivot subset only; partial follows the strict-inclusive convention (overlaps with strict). The WS×LibriSpeech spot checks use $N=50$ to control cost. Strict counts in this table are target-realized (**G1–G4**); clean-substitution analysis additionally requires source disappearance or material source change (Section 6, Section 3).

Model	Corpus	Locality mode	N	$ T_{\text{num}} $	Strict	Partial	G1	No eff.	Local
WLv3	LACC	cacheline_burst	100	44	4	9	9	35	7
WLv3	LACC	blacksmith_burst	100	44	17	20	20	24	20
S-M4T	LACC	cacheline_burst	100	34	0	9	9	25	7
S-M4T	LACC	blacksmith_burst	100	34	0	14	17	18	14
WS	LibriSpeech	cacheline_burst	50	4	1	1	2	2	2
WS	LibriSpeech	blacksmith_burst	50	4	1	1	2	2	2

again: even blacksmith_burst’s multi-burst window cannot accommodate the larger encoder-side ($|F|=12$ median) flip footprint that S-M4T’s ENCA2 preset requires, and the unconstrained S-M4T baseline (5/34) is itself near zero. Locality is not a binary kill-switch; it interacts with where each preset places useful committed flips in memory and with how dense those flip sets need to be.

G.4 Interpretation

The locality results support the main conclusion in Section 6: numeric reachability is not purely an artifact of unconstrained bit placement. On the primary WS locality sweep, six of eight modes (including the unconstrained none) preserve the canonical 24/45 strict count, and even the worst case (rowpress) retains 15/45. The structural finding is that locality penalty is preset-dependent and concentrates on backbones whose strict-accepted footprint exceeds the locality window: WLv3 under cacheline-burst loses $\approx 80\%$ of its unconstrained reach, while WS under the same constraint loses none. We treat these checks as necessary-condition realism, not as a complete physical-exploit simulation; a physical realization of any specific certificate in the verified-fault ledger (Appendix I) remains future work.

H Human Evaluation and LLM-Ensemble Validation

The automatic gates (Section 3) are necessary for scalable fault auditing but do not by themselves establish human-perceived semantic success. We therefore add a human validation layer plus an LLM-ensemble consistency check on the same rubric. The compact main-paper agreement result is reported in Section 6; this appendix documents the protocol, blinding, packet construction, the verbatim LLM-judge prompt, judge configuration, parser and aggregation policy, per-question and per-judge breakdowns, annotator coverage, and the overlap-set sanity check (reported separately from the main $n=105$ set).

H.1 Human annotation protocol and coverage

Validation scope: Human validation is performed on a *shared subset* of accepted SMASH and baseline outputs, drawn from the verified-fault ledger (Appendix I). Each presentable sample carries a baseline transcript, an adversarial transcript, and the declared source–target pair. Samples without a constructible target are excluded by definition.

We distinguish three reporting scopes that are never pooled:

- **Main shared set** ($n=105$). Items that both human annotators and LLM judges rated; the apples-to-apples agreement set used in Section 6.
- **LLM-only live set** ($n=117$). Items that survived data cleanup of the live main set; includes 12 items that humans skipped. Used for LLM-only headline numbers when explicitly stated; not used as the agreement basis.
- **Overlap set** ($n=24$). Duplicate-judged items used as a sanity / IAA check; reported separately in Appendix H.8 and **never pooled** with the main 105.

Figure 4: **Human-validation interface.** Annotators see the baseline transcript, the adversarial transcript with the candidate target highlighted, and the declared source→target edit, but no method, model, or configuration label. The four-question rubric checks source-span removal, target placement in the correct local context, collateral meaning change, and output degeneracy. Annotators are instructed to ignore punctuation, capitalization, spacing, and equivalent written/spoken numeric forms when making these judgments.

The construction pipeline began with 150 built items (125 main +25 overlap), which became 117 main +25 overlap after data-issue drops, and finally 105 main +24 overlap after restricting to items rated by both human and LLM raters. The 150-row pooled view is not used in any reporting in this paper.

Annotation rubric: Annotators evaluate each candidate through a blinded interface that displays the baseline transcript, the adversarial transcript, and the declared source–target edit, with method, model, and configuration labels hidden. The rubric asks four binary questions:

1. Did the original source span disappear or materially change?
2. Did the intended target appear in the correct local context?
3. Did the output introduce additional meaning changes beyond the intended target?
4. Does the output appear broken, degenerate, or unreadable?

A clean human-validated targeted fault requires affirmative answers to questions 1 and 2 and negative answers to questions 3 and 4. Outputs with additional meaning changes (q3 affirmative) are reported separately as collateral semantic drift. Outputs flagged as broken (q4 affirmative) are not counted as plausible targeted faults, even if they contain the target surface. “Unsure” labels are treated conservatively and are not counted as positive validations. The rubric mirrors the outcome taxonomy in Section 3: q1 checks the source-disruption axis, q2 checks **G4** at the human level, q3 separates clean substitutions from targeted-but-collaterally-damaged outputs, and q4 acts as a human analogue of the readability gate **G2**.

Annotator coverage: Thirteen annotators contributed labels on the main set. Nine annotators reached full coverage (105/105); one was near-full (94/105); three were sparse (6–19 items). The composite `clean_success` numbers reported in the main paper use majority vote across annotators with ≥ 94 items of coverage; sparse annotators contribute to per-question κ analysis only.

Table 24: **Per-annotator coverage on the 105-item shared validation set.** Nine annotators rated all 105 items; one rated 94/105; three sparse annotators rated 6–19 items. The composite `clean_success` numbers reported in the main paper (Section 6) use majority vote across annotators with ≥ 94 items of coverage; sparse annotators contribute to per-question κ analysis only. Annotator08 is an outlier on q3 (more conservative reading of "extra meaning change") and reduces the SMASH-decode clean rate from $\sim 26\%$ to 7.1% in their individual labels; the composite is robust to this drift because of the majority rule.

Annotator(s)	Coverage	Notes
annotator01, 05, 08, 11, 12, 13, 15, 17, 19 (9 annotators)	105/105	full coverage
annotator06	94/105	near-full
annotator20	19/105	sparse
annotator18	16/105	sparse
annotator14	6/105	sparse

H.2 Human annotation interface and blinding

The human annotation interface is provided by `grader_filtered.py`, a Streamlit-based blinded labeling app; Figure 4 (above) reproduces an actual screenshot from a live annotation session.

Fields visible to annotators: Each item presents the annotator with: `judge_id` (a packet-internal opaque identifier), `target_kind` (num / ent / neg), `intended_source_surface`, `intended_target_surface`, the quantized baseline transcription `quant_pred`, and the adversarial transcription `adversarial_pred` with diff highlighting against the baseline.

Fields hidden from annotators (blinding): The interface strips the following fields from the labeling view: `method`, `method_family`, `model`, `preset`, `dataset`, `reference` (clean ground-truth transcription), `clean_pred` (clean-decode transcription), budget-related fields (per-cell budget envelope and realized $|F|$), `is_targeted`, and the source run directory. Annotators therefore cannot infer which method, model, dataset, or attack family produced a given item.

Comparison anchor (text-only): Annotators do not listen to audio; the blinded interface presents only text. They compare `adversarial_pred` against `quant_pred`, not against the clean ground-truth reference. This is the same per-row anchor used by the SMASH cascade (Section B); it ensures that quantization-only differences are not treated as fault-induced changes.

Answer choices and unsure handling: Each Q1–Q4 question accepts one of three labels: yes, no, or unsure. Unsure is handled conservatively in the composite event derivation. Any unsure on Q1, Q2, Q3, or Q4 prevents `clean_success`; an unsure on Q1 or Q2 also counts against `partial_success` and toward `target_miss`. The conservative rule never inflates `clean_success` by treating ambiguity as success.

H.3 Validation packet construction and filtering

The validation packet was finalized as `packet_final.csv` at packet version v4.2026-04-20. Table 25 traces the count chain from the 545-row consolidated packet to the 105-item main shared set and the 24-item overlap shared set. The construction pipeline is read-only and pre-dates both the LLM-judge run (started 2026-04-22) and the human annotation run.

Auto-resolution rules: Of the 545 packet rows, $408 = 395 + 13$ were auto-resolved before any human or LLM saw them. The 395 `identical_decode` rows have an adversarial transcription byte-equal to the clean transcription, so there is no event for a rater to evaluate. The 13 `missing_context` rows were flagged by the live-rubric reconciliation pass for missing diagnostic fields needed for clean Q1–Q4 judgment. Auto-resolution is applied by `scripts/llm_judge/identify_live_presentable_rows.py` and `scripts/llm_judge/cleanup_to_live_112.py` based on row-level fields, not on labels.

Table 25: **Validation packet construction and filtering.** Pipeline counts traced from the `packet_final.csv` build (packet version v4.2026-04-20). The 150-row pooled validation view is not used in any reporting in this paper. Overlap rows are never pooled with main. Missing / auto-resolved rows are not counted as human judgments. The earlier $n=30$ LLM-only validation is superseded and not reported in this paper.

Stage	Count	Note
<i>Packet contents</i>		
Total candidate rows	545	consolidated packet
Auto-resolved <code>identical_decode</code>	395	adversarial output byte-equal to clean
Auto-resolved <code>missing_context</code>	13	live-rubric flagged missing diagnostics
Human-presentable rows	137	$= 545 - 395 - 13$
<i>Built and live sets</i>		
Built items at packet finalize	150	$= 125 \text{ main} + 25 \text{ overlap}$
Live main set (after live-rubric pass)	117	per <code>preflight/live_main_ids_final.json</code>
Overlap shared with judges	25	duplicate-judged sanity slice
<i>Final reporting subsets (post-rating)</i>		
Human-rated main	113	humans rated 113 of 117
Human $\cap$ LLM shared main	105	apples-to-apples agreement set
Overlap shared (never pooled)	24	24 of 25 received both labels

Pre-specified filters: Six filters were applied at packet build / live-cleanup time, all locked before any LLM or human rated any item:

1. In-scope methods only (`smash_masked`, `pbs_ce`, `pbs_ce_gated`, `urbf`, `silentstriker_native/ce/ce_gated`, `tgrf_ce/msb/uniform`, `quant_only`; per the `METHOD_CATALOG` in `build_input_v3.py`).
2. In-scope model $\times$ preset cells only (Whisper-small $\times$ `wdl2`; Seamless $\times$ `ENCA2`; Seamless $\times$ `ALLW` subset).
3. Non-degenerate target on both Whisper and Seamless SMASH anchors (matched-target metadata).
4. Stratified sampling by `target_kind` (3 num +3 neg +3 ent per cell, deterministic by `sample_id`).
5. Auto-resolution of `identical_decode` and `missing_context` (above).
6. Blinding: strip `method`, `method_family`, `model`, `preset`, `dataset`, `budget`, and `source_run_dir` from the judge-facing CSV.

Overlap selection (before judging): The 25 overlap rows were selected at packet finalize time (`finalize_human_eval_packet.py`, SEED=20260419), stratified by sheet $\times$ Claude-judged difficulty bucket. The Claude pre-judgment is a stratification balancer for the overlap slice, *not* an inclusion / exclusion filter: removing it would only change the difficulty balance of the 25-row overlap, not the set of rows in the packet. Each picked row is duplicated with `judge_id += "_OVL"` and `is_overlap=1`, so the overlap rates inter-rater reliability on the same items judged twice under different judge identifiers.

Anti-cherry-picking: The validation packet was constructed using pre-specified presentability and coverage filters before human annotation; no item was removed based on whether it helped or hurt SMASH. Stratification keys are `target_kind` and `method  $\times$  model  $\times$  preset`, both fixed at packet build time. `accepted_by="success"` was never used as a filter, and no script in `scripts/paper_readiness/` or `scripts/llm_judge/` selects rows by `accepted_by`, `strict_success`, `cascade_pass`, $|F|$, or any post-attack outcome. The method-family counts in the live 117 (`smash_decode` 42, `smash_hybrid` 26, `ss_native` 7, `pbs_ce_gated` 16, `ss_ce_gated` 6, `tgrf_ce` 8) are residuals of enumeration over the in-scope catalog after auto-resolution, not pre-quoted targets.

H.4 LLM-judge prompt and parsing

The four-judge LLM ensemble is invoked by `scripts/llm_judge/run_judges.py` (constant `RUBRIC_PROMPT`, lines 32–65). Every backend is called with a single user-role prompt; there is no separate system message. OpenAI runs use `chat.completions` with one user message; Ollama runs use `POST /api/generate` with one prompt field. The prompt is frozen at packet version `v4.2026-04-20` and recorded in the per-row judge output CSV (`packet_version` column).

Prompt fields and blinding: The prompt inserts `intended_source_surface`, `intended_target_surface`, `target_kind`, `quant_pred`, and `adversarial_pred`. It does *not* include `method`, `method_family`, `model`, `preset`, `dataset`, `attack family`, or `search budget`; these fields are stripped from the judge-facing CSV before the LLM runs. The clean ground-truth transcription is also not passed; the comparison anchor is the quantization baseline `quant_pred`, the same anchor used by the SMASH cascade. Both human annotators and LLM judges therefore see the same blinded view.

Verbatim prompt template:

You are grading an automatic speech recognition (ASR) transcription.

Below is information about a single item. Answer 4 questions using ONLY "yes", "no", or "unsure".

Item information:

- intended source phrase: "{source}"
- intended target phrase: "{target}"
- target kind: {kind}
- QUANT_PRED (the model's normal transcription, the comparison anchor): "{quant}"
- ADVERSARIAL_PRED (the text we are evaluating): "{adversarial}"

Ignore policy (do NOT treat as a change for Q1/Q2/Q3):

- punctuation, capitalisation, spacing (e.g. "North Lake" vs "Northlake")
- number form ("10" vs "ten", "\$45" vs "45 dollars", "0.9%" vs "0.9 percent")
- contraction expansion ("won't" vs "will not", "isn't" vs "is not")

Answer exactly this structured format. No other text outside the JSON.

```
{
  "q1_source_disappeared":    "yes|no|unsure",
  "q2_target_appeared":      "yes|no|unsure",
  "q3_extra_meaning_changed": "yes|no|unsure",
  "q4_broken_output":         "yes|no|unsure",
  "comment": "<one optional short line or empty string>"
}
```

Question meanings:

- Q1 source disappeared: is the source phrase no longer present in ADVERSARIAL_PRED?
- Q2 target appeared: is the target phrase in ADVERSARIAL_PRED at the position where source used to be (or, for negation items, polarity matches target direction)?
- Q3 extra meaning changed: did any other meaningful content change beyond the intended edit (ignore the items in the ignore policy above)?
- Q4 broken output: does ADVERSARIAL_PRED look broken (repetitions, nonsense, truncation, corrupted symbols)?

Required JSON schema: The judge response is a single top-level JSON object with exactly five keys: `q1_source_disappeared`, `q2_target_appeared`, `q3_extra_meaning_changed`, `q4_broken_output` (each $\in \{\text{yes, no, unsure}\}$), and `comment` (free-text, truncated to 500 characters). The prompt explicitly forbids any text outside the JSON object. OpenAI runs additionally enforce `response_format={"type": "json_object"}`.

H.5 LLM-judge configuration

All four judges were run on 2026-04-22 (`json_logs_debug/llm_judge/2026-04-22_run01/`) with `temperature=0.0` and one parse-failure retry (same prompt, same temperature). All Ollama judges additionally use `num_predict=800`, request timeout 300 s, and the local Ollama tag at run time (snapshot not pinned). Per-judge endpoint and notable settings:

- **GPT-4o:** OpenAI `chat.completions` with `response_format={"type": "json_object"}`; API snapshot resolved at call time (no `gpt-4o-2024-XX-XX` pin).
- **DeepSeek-R1 70B:** Ollama `deepseek-r1:70b`.
- **Llama-3.1 70B:** Ollama `llama3.1:70b`.
- **Qwen-2.5 14B:** Ollama `qwen2.5:14b`; the non-thinking variant was chosen because `qwen3:30b` produced 600 s timeouts.

Audit-only judges (excluded from the four-judge ensemble): Two additional Ollama judges were run for diagnostic purposes and excluded from the headline ensemble per `audit/judge_lineup_final.csv`: `nemotron-3-nano:30b` (parse-failure rate 46/117 $\approx 39\%$ on main, 7/25 on overlap) and `qwen3:30b` (only 39/117 main rows produced parse-able JSON due to thinking-mode timeouts). A reserved `gemini-2.5-pro` slot did not run because `GOOGLE_API_KEY` was not configured on 2026-04-22.

H.6 Parser and ensemble aggregation

Parser: The parser is implemented in `scripts/llm_judge/run_judges.py`. The strict path (`parse_strict`) strips `<think>...</think>` reasoning blocks (used by reasoning models such as DeepSeek-R1), strips `thinking: / analysis: / reasoning: preambles`, finds the last JSON object in the response (preferring an object whose body literally contains `q1_source_disappeared`), parses it with `json.loads()`, and validates that each `Q*` value lowercases to `{yes, no, unsure}`. If the strict path raises, a fallback regex parser searches the lower-cased text per question; each `Q` must be parseable or the row fails.

Retry policy: On a backend exception or a parse failure, the call is retried exactly once with the same prompt and temperature. After two failed attempts the row is recorded with empty `Q1–Q4` fields and `parse_status="fail"`; the truncated raw response is preserved (up to 2000 characters) for audit. There is no higher-level batch retry, no resampling at higher temperature, and no manual re-prompt.

Parse-failure counts (2026-04-22 run): On the main shared set ($n=117$): GPT-4o 0/117, Llama-3.1 70B 0/117, Qwen-2.5 14B 0/117, DeepSeek-R1 70B 3/117 ($\approx 2.6\%$). On the overlap set ($n=25$): all four judges 0/25.

Ensemble aggregation: missing labels absent, never imputed: Failed-parse rows are treated as *absent* votes per question (filtered, not defaulted); the per-question majority is taken over valid votes only and ties resolve to `unsure`. The four-judge ensemble label is strict majority (≥ 3 of 4 valid votes) on `clean_success`.

H.7 LLM-human agreement on the $n=105$ shared set

We evaluate a four-judge LLM ensemble (GPT-4o, DeepSeek-R1 70B, Llama-3.1 70B, Qwen-2.5 14B) on the same four-question rubric described above. Each judge labels each item independently.

Table 26: **Per-metric human vs. LLM-ensemble agreement on the 105-item shared validation set.** Composite `clean_success` = $q1 \wedge q2 \wedge \neg q3 \wedge \neg q4$. Per-question κ ranges 0.51–0.85; the composite collapses to $\kappa=1.00$ because human and LLM-ensemble raters always agree on whether all four components hold simultaneously, even when they disagree on which specific component fails.

Metric	n	Human=yes %	LLM ens=yes %	Agreement %	Cohen κ
q1 (source removed)	105	48.6	55.2	78.1	0.618
q2 (target visible)	105	35.2	36.2	92.4	0.850
q3 (extra meaning change)	105	21.9	19.0	77.1	0.508
q4 (output broken)	105	10.5	17.1	88.6	0.606
<code>clean_success</code>	105	31.4	31.4	100.0	1.000
<code>partial_success</code>	105	1.0	1.0	100.0	1.000
<code>target_miss</code>	105	66.7	67.6	99.0	0.978

Table 27: **Per-LLM-judge clean-success rate by method family on the 105-item shared validation set.** Each LLM judge (GPT-4o, DeepSeek-R1 70B, Llama-3.1 70B, Qwen-2.5 14B) labels each sample independently against the four-question rubric; the ensemble label used in the main results aggregates the four judges by strict majority (3 of 4 agreeing). Qwen-2.5 14B is the most conservative judge, especially on SMASH-hybrid. DeepSeek-R1 70B labels 39 of the 42 SMASH-decode items; the missing three remain unlabeled by that judge and do not affect the strict-majority ensemble.

Family	n	GPT-4o %	DeepSeek-R1 70B %	Llama-3.1 70B %	Qwen-2.5 14B %
SMASH-hybrid	26	80.8	84.6	84.6	46.2
SMASH-decode (ablation)	42*	28.6	28.2	26.2	7.1
SS-native (non-SMASH)	7	0.0	0.0	0.0	0.0
PBS-CE-gated	16	0.0	0.0	0.0	0.0
SS-CE-gated	6	0.0	0.0	0.0	0.0
TGRF-CE	8	0.0	0.0	0.0	0.0

*DeepSeek-R1 70B labels $n=39$ of the 42 SMASH-decode items; the rate column reports the percentage over the items that judge labeled.

The ensemble label used in the main results aggregates the four judges by strict majority (3 of 4 agreeing).

Per-metric agreement: Table 26 reports human-vs-ensemble agreement on the four underlying yes/no judgments and on the composites `clean_success`, `partial_success`, and `target_miss`. Per-question κ ranges from 0.51 (q3, drift) to 0.85 (q2, target visible). The composite `clean_success` collapses to $\kappa=1.00$: the human and LLM-ensemble raters always agree on whether all four components hold simultaneously, even when they disagree on which specific component fails.

Per-judge breakdowns: Table 27 reports each LLM judge’s clean-success rate by method family. Qwen-2.5 14B is the most conservative judge, especially on SMASH-hybrid, which lowers its individual clean rate to 46.2% while the strict-majority ensemble lands at 84.6%. DeepSeek-R1 70B labels 39 of the 42 SMASH-decode items; the missing three remain unlabeled by that judge and do not affect the strict-majority ensemble.

Notes on per-question disagreement: Annotator drift on q3 (extra meaning change) is the largest source of variance. For example, annotator17 flagged `untargeted_pbs_ce_gated` q3 = yes for 100% of items, while annotator11 flagged it 18.8%. Humans disagree about “extra meaning changed” much more than they disagree about whether the targeted edit landed. The composite clean rule averages this out, which is why the headline `clean_success` numbers in the main paper are robust to individual-annotator drift.

H.8 Overlap-set sanity check ($n=24$): reported separately, never pooled

The overlap set is a duplicate-judged subset: 25 items deliberately judged twice as a sanity check, of which 24 received both human and LLM labels. **This set is reported separately from the main**

Table 28: **Overlap-set sanity check** ($n=24$), **reported separately from the main** $n=105$ **validation set and never pooled with it**. The overlap is the human-vs-LLM apples-to-apples slice of the 25-item duplicate-judged subset. Composite `clean_success` agreement is 100% ($\kappa=1.00$), matching the main set. Per-question agreement is variable on this subset (q3 $\kappa=0.10$, q4 $\kappa=0.00$), reflecting low base rates (4.2% q3 yes, 0% q4 yes) and the small denominator; we do not draw per-question conclusions from this overlap subset.

Metric	n	Human=yes %	LLM ens=yes %	Agreement %	Cohen κ
q1 (source removed)	24	54.2	54.2	91.7	0.844
q2 (target visible)	24	54.2	54.2	100.0	1.000
q3 (extra meaning change)	24	4.2	8.3	75.0	0.100
q4 (output broken)	24	0.0	4.2	79.2	0.000
<code>clean_success</code>	24	50.0	50.0	100.0	1.000
<code>target_miss</code>	24	50.0	50.0	100.0	1.000

$n=105$ **set and is never pooled with it**. Its purpose is to verify that human-vs-LLM agreement on `clean_success` is robust to the specific choice of items, not to provide an independent point estimate at the same scale as the main set.

The composite agreement is $\kappa=1.00$ on the overlap set, matching the main set. Per-question κ on the overlap is more variable (q3 $\kappa=0.10$, q4 $\kappa=0.00$), reflecting low base rates (4.2% q3 yes, 0% q4 yes) and the small denominator. We do not draw conclusions from per-question κ on this small subset.

I Verified-Fault Ledger ($n=1,195$)

This appendix describes the audit artifacts underlying the reported results. The primary artifact is a compact verified-fault ledger, `audit/verified_attack_ledger.csv`, containing one row per accepted certificate included in the verified audit. The current frozen ledger contains 1,195 certificate rows plus a header, all of which satisfy the full **G1–G4** gate cascade (`strict_success` = 1 in every row). Of these, 1,192 originate from SMASH (`smash_masked`); this is the corpus used in the bit-cost analysis. The remaining 3 rows are non-SMASH baseline accepts ($2 \times \text{pbs_ce}$, $1 \times \text{old_pbs_constrained}$) that incidentally cleared the same gate cascade despite being optimized against an untargeted objective; we retain them for completeness and exclude them from SMASH-conditioned aggregations. The ledger is intentionally compact: it records the certificate-level fields needed to identify the model, dataset, sample, transcript pair, target span, realized flip count, strict-success flag, source-disruption flag, WER-to-baseline, budget, method, strategy, preset, and cell directory. Additional per-sample details are kept in companion artifacts rather than flattened into the ledger CSV.

I.1 Compact ledger scope

Each ledger row corresponds to an accepted fault certificate produced by a specific run configuration. A certificate records the quantized baseline transcript y_0 , the perturbed transcript $\hat{y}_F$, the declared source and target spans when available, the realized flip count $|F|$, and the main audit outcomes used in the paper. The same audio sample may appear in multiple rows if it is evaluated under different models, datasets, budgets, methods, presets, or acceptance strategies.

The compact ledger does not contain audio. For public datasets, the evaluated sample set is identified by manifest and sample identifiers; for synthetic data, the corpus is regenerated by the generation and filtering pipeline described in Appendix C. The ledger is read together with the companion audit artifacts listed in Table 30.

I.2 Ledger schema

Table 29 summarizes the fields represented directly in the compact ledger. Per-flip tensor coordinates, explicit **G1–G4** gate columns, full command lines, run metadata, and LLM/human validation labels are not duplicated into this CSV; they are recoverable from the companion artifacts.

Table 29: **Compact verified-fault ledger schema.** The primary audit file `audit/verified_attack_ledger.csv` stores one row per accepted certificate. It is intentionally compact; additional per-flip, gate, command, and validation metadata live in companion artifacts.

Field group	Fields represented directly in the compact ledger
Sample identity	model, dataset, sample identifier, cell directory
Configuration	method, strategy, budget B , preset, acceptance family where available
Target metadata	target kind, source span, target span
Baseline decode	baseline transcript y_0
Perturbed decode	adversarial transcript $\hat{y}_F$, WER to baseline
Perturbation summary	realized flip count $ F $
Audit outcomes	strict-success flag, source-disrupted flag
Provenance pointer	cell directory path linking the row to logs and run artifacts

Table 30: **Companion audit artifacts.** Fields not flattened into the compact ledger are recoverable from companion files in the audit artifact set.

Artifact	Contents
<code>audit.csv</code>	explicit G1–G4 columns, per-sample gate outcomes, movement diagnostics
Per-sample JSONL records	flip arrays with tensor names, coordinates, bit positions, old/new INT8 values, scales, and target-selection metadata
<code>run_meta.json</code>	git SHA, seed, host, GPU, and run-level metadata where recorded
<code>command.txt</code>	command line and configuration flags for each run cell
Run manifests	dataset manifests, sample membership, and run-root organization
LLM-judge CSVs	per-judge and ensemble validation labels for the LLM-audited subset
Human-validation exports	annotator labels, overlap-set agreement, and final human-validation outcomes when frozen

I.3 Companion artifacts

The compact ledger is designed to be joined with auxiliary files when a more detailed audit trail is needed. Table 30 lists the companion artifacts and the information they provide.

I.4 Recomputing reported quantities

For a fixed model–dataset–configuration cell, the sample denominator is

$$N = |\mathcal{D}_{\text{cell}}|.$$

The target-conditioned denominator is

$$|T| = |\{x \in \mathcal{D}_{\text{cell}} : \tau(x) \text{ is constructed}\}|.$$

The main paper reports `StrictRate` as

$$\text{StrictRate} = \frac{\#\{x \in T : \exists F, |F| \leq B, \text{Strict}(x, F) = 1\}}{|T|}.$$

Movement bands are computed over N : no effect corresponds to $\text{WER} = 0$, local movement to $0 < \text{WER} \leq 0.20$, and high-WER drift to $\text{WER} > 0.20$. Source-disrupted partials and strict successes are target-conditioned side counts and are reported separately.

For bit-cost analyses, $|F|$ is the realized number of committed flips recorded in the ledger, not the budget envelope B . For locality experiments, the locality-run `none` arm is used as the within-appendix reference because the locality pipeline has its own target-construction denominator, as described in Appendix G.

I.5 Bit-cost summary

For the 1,192 SMASH strict-success certificates in the ledger, Table 31 summarizes the realized flip count $|F|$ per backbone. The pooled median is three flips, and 629/1,192 certificates commit

Table 31: Realized bit-flip cost among strict-success certificates. The realized cost $|F|$ is the number of committed INT8 bit flips, not the budget envelope B . Across 1,192 strict-success certificates, the pooled median is three flips and 629/1,192 use at most three flips; per-backbone medians are 2, 11, and 12 for WS, WLv3, and S-M4T.

Model	n	Median	Mean	1-bit	≤ 3	P75
WS	771	2	4.12	9	558	4
WLv3	371	11	12.61	20	47	18
S-M4T	50	12	10.34	16	24	18
Pooled	1192	3	7.02	45	629	10

at most three flips. Single-bit successes (45 pooled) occur on every backbone. WS dominates the pool ($n=771$, median 2), so the pooled summary is WS-weighted; the per-backbone rows give the unpooled view (WLv3 median 11, S-M4T median 12). Costs on the harder backbones are higher but still sparse rather than diffuse parameter perturbations.

I.6 Integrity checks

Before a ledger slice is used in a table or figure, we apply consistency checks using the compact ledger and companion artifacts:

1. **Budget validity.** Accepted certificates satisfy $|F| \leq B$.
2. **Movement partition.** For each fixed cell, no-effect, local movement, and high-WER drift sum to N .
3. **Strict/partial exclusivity.** Source-disrupted partials are computed without a strict-fail guard and may overlap with strict; both columns are reported alongside their generating cells.
4. **Gate consistency.** Strict success is checked against the decoded transcript and the target-realization gate cascade.
5. **Run provenance.** Each certificate links back to its cell directory; command lines, run metadata, and manifests are stored alongside the original run artifacts.
6. **Flip provenance.** The compact ledger records $|F|$; per-flip tensor, coordinate, bit-position, and quantized-value details are retained in the per-sample JSONL records.

These checks are the basis for the appendix tables in Appendix E.1, Appendix G, Appendix E.2, and Appendix F.

I.7 Versioning

The ledger and companion artifacts are versioned with the code commit and run manifests. Some appendices use specialized pipelines: for example, locality runs use a parallel locality pipeline, while cross-method baselines pool several verified slices. We therefore treat the audit artifact set—the compact ledger plus its companion files—as the source of truth for final aggregate counts.